KLE Technological University | Creating Value, Leveraging Knowledge

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

A Senior Design Project Report on

AI-Powered Lung Disease Detection System

submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

Submitted by

Sharan Pachapuri	01FE23BCS027
Sampad Gaonkar	01FE23BCS024
Vijet Nayak	01FE22BCS324

Under the guidance of

Prof. Indira Bidari
Assistant Professor
School of Computer Science and Engineering

KLE Technological University, Hubballi

2026–2027

KLE Technological University

Creating Value, Leveraging Knowledge

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

2026–2027

CERTIFICATE

This is to certify that the project entitled **"AI-Powered Lung Disease Detection System"** is a bona fide work carried out by the student team **Sharan Pachapuri** (01FE23BCS027), **Sampad Gaonkar** (01FE23BCS024) and **Vijet Nayak** (01FE22BCS324), in partial fulfilment of the 7th semester Senior Design Project (20ECSW401) during the academic year 2026–2027. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the above-mentioned course.

Prof. Indira Bidari	**Dr. Vijayalakshmi M.**
Guide	Head, SoCSE

External Viva-Voce

Name of the Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

ABSTRACT

Chest X-rays and chest computed tomography (CT) scans are the most widely used imaging tests for lung disease, yet India has roughly one radiologist for every one hundred thousand people, most of them in large cities. Deep-learning classifiers can assist with triage, but most published systems handle a single imaging modality, return only a label with no explanation, and confidently label any image they are given, including photographs and scans of the wrong type.

This project presents the *AI-Powered Lung Disease Detection System*, a full-stack web application that classifies chest X-rays into four classes (COVID-19, Normal, Pneumonia and Tuberculosis) and chest CT slices into three classes (Benign, Malignant and Normal) using transfer-learned DenseNet-121 networks. Before any disease model runs, every upload passes a chain of checks: basic image validation, a per-modality sanity gate that rejects non-scans, a ResNet-18 modality gate that rejects the wrong scan type, and a feature-space out-of-distribution detector. X-rays are additionally cropped to the lung fields by a U-Net segmentation model. Every prediction is returned with a Grad-CAM heatmap, a plain-language report generated by a large language model under a strict non-diagnostic prompt, and access to a scoped chat assistant. The application also provides email and Google sign-in, scan history, rate limiting and a public HTTPS deployment on a cloud server, and a start-up check can re-verify all six model checkpoints against their validation data before they are served.

Data audits removed near-duplicate CT slices that had inflated test accuracy to 100% and cleaned burned-in artifacts from the tuberculosis images. On internal validation data the X-ray and CT classifiers reach 96.5% and 99.0% accuracy (96.57% and 98.06% on the held-out test splits), and through the live API the X-ray route delivers 91.05%. On new external images sent through the deployed routes, detection accuracy is 93.9% for chest X-ray (31 of 33 images accepted by the guards) and 91.9% for lung CT.

Keywords: *Chest X-ray, CT scan, deep learning, transfer learning, DenseNet-121, Grad-CAM, explainable AI, out-of-distribution detection, FastAPI, React.*

ACKNOWLEDGEMENT

We would like to thank our faculty and management for their professional guidance towards the completion of this project. We take this opportunity to thank Dr. Ashok Shettar, Pro-Chancellor, Dr. P. G. Tewari, Vice-Chancellor, and Dr. B. S. Anami, Registrar, for their vision and support.

We also thank Dr. Meena S. M., Professor and Dean of Faculty, SoCSE, and Dr. Vijayalakshmi M., Professor and Head of CSE, for providing direction and for facilitating the enhancement of our skills and academic growth.

We thank our guide, Prof. Indira Bidari, Assistant Professor, School of Computer Science & Engineering, for her constant guidance during our interactions and reviews, and for encouraging us to question our own results rather than accept the first accuracy figure we obtained.

We extend our acknowledgement to the reviewers for their critical suggestions and inputs, and to the project coordinators, Dr. Uday Kulkarni and Mr. Sunil V. Gurlahosur, for their suggestions during the course of the project. We are grateful to the authors of the public COVID-19 Radiography, Tuberculosis Chest X-ray and IQ-OTH/NCCD datasets, and to the open-source communities behind PyTorch, TorchXRayVision, FastAPI and React, whose work made this project possible. Finally, we express our gratitude to our parents for their constant encouragement and support.

Sharan Pachapuri	01FE23BCS027
Sampad Gaonkar	01FE23BCS024
Vijet Nayak	01FE22BCS324

CONTENTS

ABSTRACT i

ACKNOWLEDGEMENT ii

CONTENTS iii

LIST OF TABLES vii

LIST OF FIGURES ix

1 INTRODUCTION 1

1.1 Preamble 1

1.2 Motivation 2

1.3 Literature Study 2

1.4 Problem Definition 4

1.5 Objectives of the Project 4

1.6 Organisation of the Report 5

2 SOFTWARE REQUIREMENT SPECIFICATION 6

2.1 Overview of SRS 6

2.2 Requirement Specifications 6

2.2.1 Functional Requirements 6

2.2.2 Use Case Diagram 7

2.2.3 Use Case Descriptions Using Scenarios 7

2.2.4 Non-Functional Requirements 12

2.3 Software and Hardware Requirement Specifications 13

3 PROPOSED SYSTEM 15

3.1 Description of the Proposed System 15

3.2 Description of Target Users 16

3.3 Advantages and Applications of the Proposed System 16

3.3.1 Design Principles . . . 16
3.3.2 Advantages . . . 17
3.3.3 Applications . . . 17
3.4 Scope (Boundary of the Proposed System) . . . 17
3.4.1 In Scope . . . 17
3.4.2 Out of Scope . . . 18
3.4.3 Assumptions and Constraints . . . 18

4 SYSTEM DESIGN 19

4.1 Architecture of the System . . . 19
4.2 Selection of Design Alternatives . . . 21
4.3 Class Diagram . . . 22
4.4 Design Patterns . . . 23
4.5 Sequence Diagram . . . 25
4.6 ER Diagram and Schema . . . 25
4.7 State Transition Diagram . . . 26
4.8 Dataset Description . . . 27

5 IMPLEMENTATION 30

5.1 Proposed Methodology . . . 30
5.2 Algorithm Selection for Each Module . . . 31
5.3 Description of Modules . . . 32
5.3.1 Module 1: Data Preparation and Audit . . . 32
5.3.2 Module 2: Disease Classifier Training . . . 33
5.3.3 Module 3: Sanity Gate and Modality Gate . . . 34
5.3.4 Module 4: Lung Segmentation and Crop . . . 36
5.3.5 Module 5: Out-of-Distribution Detector . . . 37
5.3.6 Module 6: Classification and Grad-CAM . . . 37
5.3.7 Module 7: Report Generation and Chat Assistant . . . 38
5.3.8 Module 8: Start-up Reproduction Check . . . 38
5.3.9 Module 9: Authentication, Sessions and Access Control . . . 39
5.3.10 Module 10: Rate Limiting and Error Handling . . . 39

5.3.11 Module 11: Front End . . . 40
5.3.12 Module 12: Deployment . . . 40

6 TESTING **42**

6.1 Test Plan and Test Cases . . . 42
6.1.1 Unit and Integration Test Plan and Test Cases . . . 42
6.1.2 Guardrail Test Cases . . . 45
6.1.3 Concurrency and Model-Verification Tests . . . 45
6.1.4 Performance Testing . . . 46
6.1.5 Acceptance Test Plan and Test Cases . . . 46
6.1.6 Limitations of the Testing . . . 47

7 RESULTS & DISCUSSIONS **49**

7.1 Training Behaviour . . . 49
7.2 Classification Performance on Held-out Test Data . . . 50
7.3 Effect of the Data Audits . . . 51
7.4 Live-Serving Accuracy and the Crop Fix . . . 52
7.4.1 Segmentation-Crop Trade-off . . . 53
7.5 Internal vs. External Accuracy . . . 54
7.6 Snapshots of the Application . . . 54
7.7 Discussion . . . 58

8 CONCLUSIONS AND FUTURE SCOPE **59**

8.1 Conclusion . . . 59
8.2 Future Scope . . . 59

REFERENCES **61**

Appendix A SUPPLEMENTARY MATERIAL **64**

A.1 Screenshots . . . 64
A.2 Gantt Chart . . . 66
A.3 Glossary . . . 67
A.4 Description of Tools and Technologies Used . . . 68

A.5 Blueprint 69
A.6 Plagiarism Report 69

LIST OF TABLES

1.1 Summary of the literature survey 4

2.1 Functional requirements 7
2.2 Use case UC1: Upload and analyse a scan 9
2.3 Use case UC2: Register and verify email 10
2.4 Use case UC3: Chat with the assistant 11
2.5 Use case UC4: View scan history 11
2.6 Non-functional requirements 12
2.7 Software requirements 14
2.8 Hardware requirements 14

4.1 Weighted decision matrix for the disease-classifier backbone 22
4.2 Other design decisions and the criterion behind each 22
4.3 Design patterns applied in the system 24
4.4 Schema of the `scans` table 26
4.5 Datasets used 28
4.6 Final train / validation / test splits (random seed 42) 28

5.1 Existing algorithms identified and the algorithm selected for each module 32
5.2 Training configuration of the disease classifiers 35
5.3 REST API of the back end 39

6.1 Test plan 42
6.2 Representative unit and integration test cases 43
6.3 Guardrail test cases (live HTTP) 45
6.4 Start-up reproduction check (live start-up) 46
6.5 Measured latency (seconds) 46
6.6 Acceptance test cases 46

7.1 X-ray classifier on the test split ($n = 1{,}922$; accuracy 96.57%) 50

7.2 CT classifier on the test split ($n = 103$; accuracy 98.06%) 50
7.3 X-ray accuracy before and after the crop fix 52
7.4 Internal (validation) vs. external detection accuracy 54

A.1 Glossary of terms 67
A.2 Tools and technologies used 68

LIST OF FIGURES

2.1 Use case diagram of the system (modelled in PlantUML) 8

3.1 Framework of the proposed system 15

4.1 Three-tier architecture of the system 19
4.2 Class diagram of the main back-end classes (PlantUML) 23
4.3 Design patterns on the path of a prediction request (PlantUML component diagram) 24
4.4 Sequence diagram for analysing a chest X-ray (PlantUML) 25
4.5 ER diagram of the database 26
4.6 State transition diagram of the user interface (PlantUML) 27
4.7 Class distribution of the final X-ray and CT datasets by split 28
4.8 Sample image / lung-mask pairs from the segmentation dataset (test split) 29

5.1 Methodology followed in the project 31
5.2 Random sample of raw Tuberculosis images before cleaning, showing colour tints and white redaction boxes 34
5.3 Lung-crop pipeline on a real X-ray: (a) input, (b) U-Net mask, (c) segmentation crop, (d) border-crop fallback 36

7.1 Training and validation loss and accuracy of the CT classifier (shaded: frozen-backbone warm-up) 49
7.2 Training curves of the modality gate (ResNet-18) 50
7.3 Confusion matrices on the held-out test splits (counts and row percentages) . . . 51
7.4 Per-class precision, recall and F1-score on the test splits 51
7.5 X-ray accuracy offline and through the live route, before and after the crop fix . . 52
7.6 X-ray test split through the live API with all guards active (rejections shown as a column) 53
7.7 Detection accuracy on internal validation images and on new external images . . 54
7.8 Landing page showing the four checks every upload passes before it is classified . 55
7.9 X-ray result: predicted class, probability bars, Grad-CAM heatmap and plain-language explanation 55

7.10 CT result for a Malignant prediction, in the same layout as the X-ray result 56
7.11 Chat assistant answering “What does the heatmap show?” 56
7.12 History page with every saved scan 57
7.13 A guard firing live: a CT slice uploaded on the X-ray tab is refused with a reason 57

A.1 Login screen, with email and password or “Continue with Google” 64
A.2 Analyse screen: choosing the scan type and uploading an image 65
A.3 Analysing screen with the stage checklist 65
A.4 X-ray result with Grad-CAM heatmap (Pneumonia prediction) 66
A.5 An opened history entry showing the saved heatmap and explanation 66
A.6 Gantt chart of the project 67

CHAPTER 1

INTRODUCTION

Lung diseases such as tuberculosis, pneumonia, COVID-19 and lung cancer remain among the leading causes of illness and death worldwide, and India carries a disproportionate share of that burden. Two imaging tests are central to diagnosing them: the chest radiograph (X-ray), which is inexpensive and widely available, and the computed tomography (CT) scan, which shows the lung in cross-section and is the standard tool for characterising nodules and tumours. Reading these images well requires a trained radiologist, and radiologists are scarce outside large cities. Deep convolutional neural networks (CNNs) have been shown to match radiologists on narrowly defined image-classification tasks [1, 2], which has made automated screening and decision support an active area of applied research.

This report describes the design, implementation and evaluation of the *AI-Powered Lung Disease Detection System*, a web application developed as our Senior Design Project. A user uploads a chest X-ray or a chest CT slice; the system checks that the upload is a genuine scan of the right type, predicts the most likely condition, highlights the image regions that drove the prediction, and explains the result in plain language. The rest of this chapter introduces the project, the motivation behind it, the literature it builds on, the problem it addresses and its objectives.

1.1 Preamble

The system supports two imaging modalities in a single application. Chest X-rays are classified into four classes (COVID-19, Normal, Pneumonia and Tuberculosis) and chest CT slices into three classes (Benign, Malignant and Normal). Both classifiers are DenseNet-121 networks [3] adapted to the task by transfer learning: the X-ray model starts from weights pre-trained on a large collection of chest radiographs [4], and the CT model from ImageNet weights [5].

A plain image classifier assigns one of its known labels to *any* image it is given, including a photograph, a screenshot or a scan of the wrong kind, and it often does so with high confidence. In our own early testing, the CT model labelled a screenshot "Malignant" with 93% confidence. The system therefore places a chain of checks in front of every classifier: basic image validation; a *sanity gate* that decides whether the upload is a medical scan at all; a *modality gate* that decides whether it is an X-ray or a CT slice; and an *out-of-distribution (OOD) detector* that rejects images unlike anything seen during training. X-rays are also cropped to the lung fields by a U-Net segmentation model [6] before they are classified.

Every prediction is accompanied by a Grad-CAM heatmap [7], per-class probabilities and a short report written by a large language model (LLM) that is instructed never to diagnose or recommend

treatment. A chat assistant answers follow-up questions about the result and the tool. Around this core sits a complete web application: a React front end, a FastAPI back end, user accounts with email verification and Google sign-in, saved scan history, rate limiting and a public deployment.

A large part of the project's effort went into checking that the reported numbers mean what they appear to mean. We audited the datasets for duplicated images, burned-in artifacts and patient leakage between the training and test splits; we re-verify every model checkpoint against its validation data before it is served; we measured accuracy through the live API, not just offline; and we tested the models on images from independent sources. Several of these checks changed our results; Chapter 7 reports internal accuracy of 96.5% (X-ray) and 99.0% (CT) and external accuracy of 93.9% and 91.9%.

1.2 Motivation

The project is motivated by four observations.

- **Disease burden.** India accounts for about a quarter of the world's tuberculosis cases [8], and 80–85% of lung cancer patients in India are diagnosed when the disease is already at an advanced, incurable stage [9]. Earlier and wider screening directly affects outcomes for both diseases.
- **Shortage of radiologists.** India has roughly one radiologist per one hundred thousand people, against about one per ten thousand in the United States [10]. A screening aid that flags likely abnormal scans for priority review can stretch scarce expert time further.
- **The trust gap.** Most published chest-imaging models report very high accuracy but return only a label. A clinician cannot act on, or argue with, a label that comes with no explanation. Visual explanations such as Grad-CAM and plain-language reports make a model's output something that can be inspected and questioned.
- **Offline-only evaluation.** Models trained on public datasets are usually evaluated only on test images drawn from the same sources as their training images. A tool that is meant to be trusted should also be checked through its deployed path and on new images.

The project also aligns with United Nations Sustainable Development Goal 3, *Good Health and Well-Being* [11], specifically Target 3.3 (ending the tuberculosis epidemic) and Target 3.8 (access to quality essential health-care services), as a decision-support aid rather than a replacement for clinical diagnosis.

1.3 Literature Study

Chest X-ray classification. Rajpurkar et al. [1] trained CheXNet, a 121-layer DenseNet, on the ChestX-ray14 dataset and reported pneumonia detection at a level comparable to practising radiologists (F1-score of 0.435 against a radiologist average of 0.387). CheXpert [2] extended

this to a large multi-label benchmark with structured uncertainty labels, and COVID-Net [12] adapted a similar approach to COVID-19 screening. Closest to our datasets, Chowdhury et al. [13] screened viral and COVID-19 pneumonia on the COVID-19 Radiography Database, and Rahman et al. [14] detected tuberculosis with a pipeline that first segments the lungs and then classifies, a design that directly parallels our own U-Net lung crop. Mirugwe et al. [15] compared six ImageNet-pretrained CNNs on the same tuberculosis database and found that the simplest, VGG-16, gave the best balance of accuracy (99.4%) and computational cost. Cohen et al. [4] released TorchXRayVision, a library of chest X-ray datasets and models pre-trained on a large aggregate of radiographs, whose DenseNet-121 weights we use for our X-ray classifier.

CT-based lung cancer classification. A separate line of work classifies CT slices or volumes into benign and malignant findings; Monkam et al. [16] survey CNN methods for pulmonary nodule detection and classification. Benamara et al. [17] recently reported 99.54% accuracy on the IQ-OTH/NCCD dataset [18] with a multi-level fine-tuned VGG-16. Such studies typically use small, single-institution datasets and rarely report a check for near-duplicate slices across the training and test splits, or an evaluation on data from another hospital.

Out-of-distribution detection and explainability. Hendrycks and Gimpel [19] showed that the maximum softmax probability of a classifier is a weak signal for detecting inputs unlike its training data; our own testing reproduced this. Grad-CAM [7] produces class-specific heatmaps from the gradients flowing into the final convolutional layer and is widely used to make CNN predictions inspectable. Liu et al. [20] survey the automatic generation of medical-imaging reports; our report and chat components instead use a tightly constrained LLM prompt that forbids diagnostic or treatment language.

Table 1.1 summarises the most relevant studies and the gap that this project targets.

Identified gap. None of the surveyed systems combines two imaging modalities behind a shared layer that rejects non-scans and wrong-modality uploads, and few report a reproducible audit of data leakage or an evaluation on independent data. Most stop at a research notebook rather than a usable application with explanations.

Table 1.1: Summary of the literature survey

Study	Year	Technique	Strength	Limitation
Rajpurkar et al. (CheXNet) [1]	2017	DenseNet-121 on ChestX-ray14	Exceeded average radiologist F1-score on pneumonia	Single task; no deployed interface
Chowdhury et al. [13]	2020	CNNs on the COVID-19 Radiography Database	97.9% accuracy (COVID-19, viral pneumonia, Normal)	No explanation output; single source
Rahman et al. [14]	2020	Lung segmentation + CNN + Grad-CAM	TB localisation with visualisation	Tuberculosis only
Mirugwe et al. [15]	2025	VGG / ResNet / Inception transfer learning	VGG-16 gives the best accuracy–cost trade-off (99.4% accuracy)	Binary (TB vs. Normal); 4,200 images
Benamara et al. [17]	2025	Multi-level fine-tuned VGG-16 on CT	99.54% on IQ-OTH/NCCD	CT only; no leakage audit or external test
This project	2026	DenseNet-121 (two modalities) + gates + OOD detector + U-Net + Grad-CAM + LLM report	Two modalities, input guards, audited data, live and external evaluation	Small, single-institution CT dataset (Chapter 7)

1.4 Problem Definition

To design, implement and evaluate a dual-modality deep-learning system that

- classifies a chest X-ray into one of four classes (COVID-19, Normal, Pneumonia, Tuberculosis) and a chest CT slice into one of three classes (Benign, Malignant, Normal);
- refuses inputs it was not trained to handle, including non-medical images, the wrong scan type and out-of-distribution images;
- explains every prediction with a visual heatmap and a plain-language, non-diagnostic report; and
- is delivered as a secure, multi-user web application whose reported accuracy has been verified against the data and the deployed code path.

1.5 Objectives of the Project

The objectives of the project are:

1. To classify chest X-rays (COVID-19, Normal, Pneumonia, Tuberculosis) and chest CT slices (Benign, Malignant, Normal) using transfer-learned CNNs trained on public datasets.
2. To make every prediction explainable through a Grad-CAM heatmap, per-class probabilities, an automatically generated plain-language report and a follow-up chat assistant.

3. To reject inputs the models cannot handle: non-scan images, scans of the wrong modality and out-of-distribution images.
4. To build a full-stack web application with user registration, email verification, Google sign-in, saved scan history and protection against abuse.
5. To audit the training data for leakage and duplicates, and to evaluate the deployed system on live requests and on external data.

1.6 Organisation of the Report

The remainder of this report is organised as follows.

- **Chapter 2** specifies the software requirements.
- **Chapter 3** describes the proposed system, its target users, advantages and scope.
- **Chapter 4** presents the system design: the architecture and the class, sequence, ER and state diagrams, together with the datasets.
- **Chapter 5** explains the methodology and the implementation of each module.
- **Chapter 6** describes the test plan and test cases.
- **Chapter 7** presents and discusses the results.
- **Chapter 8** concludes the report and outlines future work.

Screenshots, the project timeline, a glossary and the tools used are given in Appendix A.

CHAPTER 2

SOFTWARE REQUIREMENT SPECIFICATION

This chapter specifies what the system must do and the qualities it must have. It lists the functional requirements, models the interactions between users and the system with a use case diagram and use case descriptions, states the non-functional requirements, and gives the software and hardware needed to build and run the system.

2.1 Overview of SRS

The Software Requirement Specification (SRS) is the agreed description of the system's behaviour and constraints against which the design, implementation and testing in later chapters are checked. The system has two kinds of human user:

- a *guest*, who can view the landing page, register, log in and reset a forgotten password; and
- a *registered, verified user*, who can analyse scans, read results, chat with the assistant and view their scan history.

The system also interacts with three external services: the Groq LLM API for reports and chat, Gmail SMTP for verification and password-reset emails, and Google OAuth for sign-in. The system is a research prototype for decision support. It is not a certified medical device and does not produce a diagnosis; this constraint shapes several of the requirements below.

2.2 Requirement Specifications

Requirements are divided into *functional requirements*, which state what the system shall do in response to inputs, and *non-functional requirements*, which state how well it shall do it (performance, security, usability, reliability and so on).

2.2.1 Functional Requirements

A functional requirement describes a specific behaviour or function of the system: the inputs it accepts, the processing it performs and the outputs it produces. Table 2.1 lists the functional requirements of the system.

Table 2.1: Functional requirements

ID	Requirement: the system shall ...
FR1	accept a chest X-ray or chest CT image in a common raster format (PNG or JPEG) of at most 10 MB, uploaded through the web interface to a modality-specific route.
FR2	reject uploads that are empty, not decodable as images, blank, or colour photographs, with a clear message.
FR3	reject uploads that are not medical scans (sanity gate), and scans that do not match the selected modality or whose modality is uncertain (modality gate).
FR4	reject images whose features lie outside the distribution of the training data (OOD detector).
FR5	classify an accepted chest X-ray as COVID-19, Normal, Pneumonia or Tuberculosis, and an accepted chest CT slice as Benign, Malignant or Normal, and report the confidence and the full probability vector.
FR6	crop chest X-rays to the lung fields using a segmentation model before classification.
FR7	generate a Grad-CAM heatmap for every prediction and let the user toggle between the heatmap and the original image.
FR8	generate a plain-language, non-diagnostic report for every prediction, falling back to a fixed template when the LLM is unavailable, and warn the user when confidence is low.
FR9	provide a chat assistant that answers questions about the current result and the tool, and declines medical-advice and off-topic requests.
FR10	let users register with an email address and password, verify their email through a time-limited link, log in with email or Google, reset a forgotten password and change their password.
FR11	store each signed-in user's predictions and display their scan history, including the saved heatmap, class, confidence and report.
FR12	in its default (strict) mode, refuse to start if any model checkpoint's claimed validation metric cannot be reproduced on the validation data.

2.2.2 Use Case Diagram

Figure 2.1 shows the use case diagram. A registered user inherits every use case available to a guest. "Upload & analyse scan" always includes "Validate scan", and the external services appear as secondary actors.

2.2.3 Use Case Descriptions Using Scenarios

The principal use cases are described in Tables 2.2 to 2.5 using the use case template of Pressman [21].

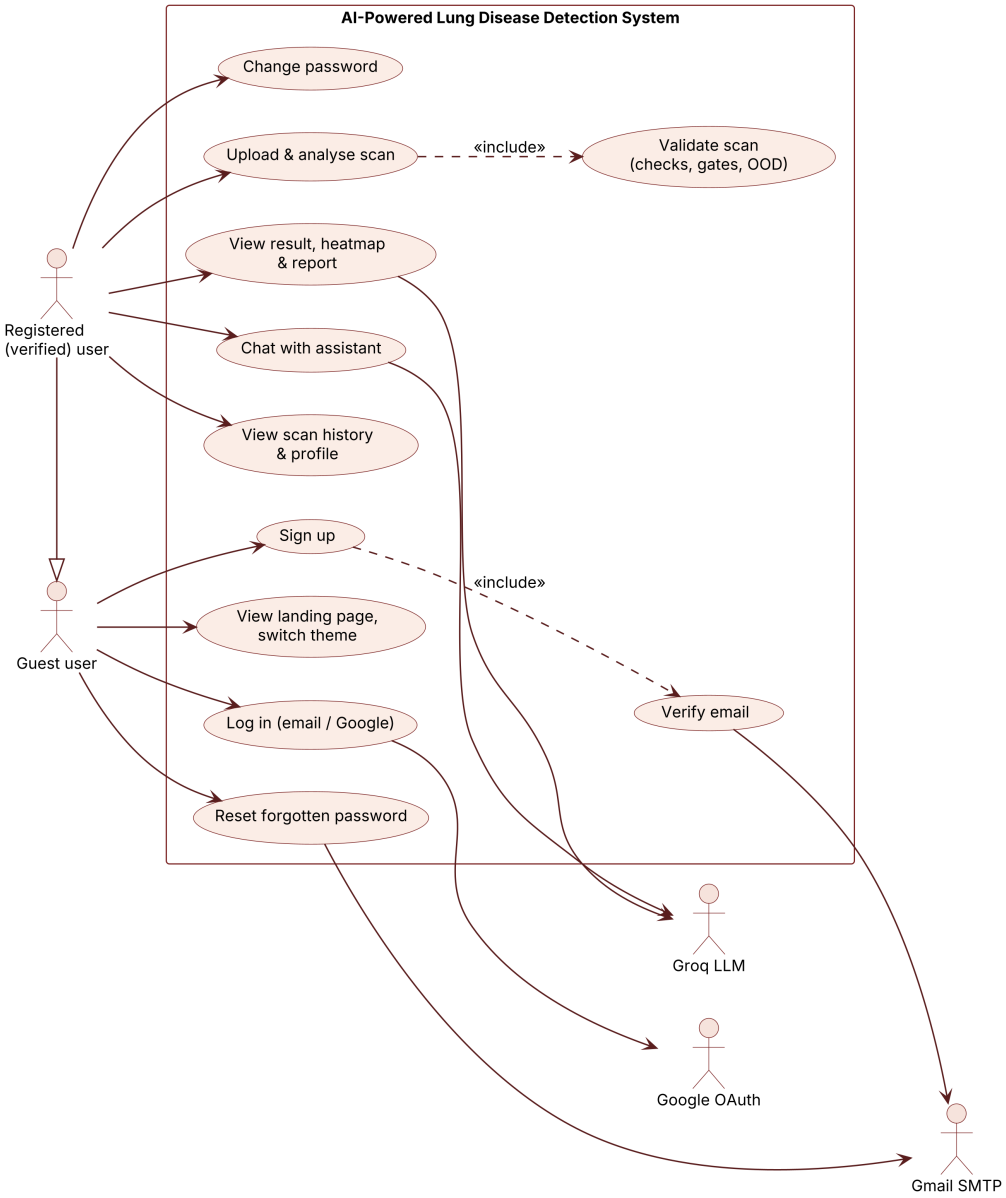


Figure 2.1: Use case diagram of the system (modelled in PlantUML)

Table 2.2: Use case UC1: Upload and analyse a scan

Use case	Upload and analyse a scan
Primary actor	Registered (verified) user
Goal in context	Obtain a predicted class, confidence, heatmap and report for a chest X-ray or CT image
Preconditions	The user is logged in and the email address is verified; the back end is running with all models loaded and verified
Trigger	The user selects a modality, chooses an image and clicks the *Analyse* button
Scenario	1. The user selects "Chest X-ray" or "Chest CT Scan". 2. The user drags and drops, or browses for, an image. 3. The user clicks *Analyse*; the analysing screen shows a progress checklist. 4. The system validates the image and runs the sanity gate, the modality gate and the OOD check. 5. The system crops X-rays to the lungs, classifies the image and computes the Grad-CAM heatmap. 6. The system generates the report and saves the scan to the user's history. 7. The results page shows the class, confidence, probability bars, heatmap and report.
Exceptions	File too large (413); not an image, empty, blank or colour (400); not a scan, wrong or uncertain modality, or out of distribution (422); low confidence (result shown with a warning); LLM unavailable (template report); back end unreachable (error with *Try again*)
Priority	Essential; highest priority
When available	First release
Frequency of use	Many times per session
Channel to actor	Web browser (React single-page application)
Secondary actors	Groq LLM service
Channels to secondary actors	HTTPS REST API
Open issues	The CT model is trained on a small single-institution dataset (Chapter 7); the result is decision support only

Table 2.3: Use case UC2: Register and verify email

Use case	Register and verify email
Primary actor	Guest user
Goal in context	Create an account and confirm ownership of the email address
Preconditions	The email address is not already registered
Trigger	The guest clicks “Create account” on the landing page
Scenario	1. The guest enters a name, an email address and a password (twice). 2. The system validates the email format and the password length (at least 8 characters). 3. The system stores the user with a bcrypt password hash and creates a single-use verification token valid for 24 hours. 4. The system emails the verification link. 5. The user clicks the link; the system marks the email as verified and redirects to the “Email verified” page. 6. The user continues to the application.
Exceptions	Email already registered (409); invalid email or short password (422); link expired, reused or unknown (400); SMTP not configured (link printed to the server console)
Priority	Essential
When available	First release
Frequency of use	Once per user
Channel to actor	Web browser and email client
Secondary actors	Gmail SMTP service
Channels to secondary actors	SMTP over TLS
Open issues	Google sign-in links to an existing account with the same email address

Table 2.4: Use case UC3: Chat with the assistant

Use case	Chat with the assistant
Primary actor	Registered (verified) user
Goal in context	Understand the prediction, the heatmap or how the tool works
Preconditions	The user is logged in; optionally, a result is loaded
Trigger	The user opens the chat widget and types a question or taps a quick-reply chip
Scenario	1. The user opens the floating chat widget. 2. The user sends a message. 3. The system sends the system prompt, the current prediction context and the last ten turns to the LLM. 4. The system returns the reply, which the widget displays.
Exceptions	Empty message (400); message over 2,000 characters or history over 50 items (422); more than 20 messages per minute (429); LLM unavailable (fixed fallback reply); medical-advice or off-topic question (the assistant declines)
Priority	High
When available	First release
Frequency of use	Several times per result
Channel to actor	Web browser
Secondary actors	Groq LLM service
Channels to secondary actors	HTTPS REST API
Open issues	LLM replies cannot be fully guaranteed; the prompt is the safety boundary

Table 2.5: Use case UC4: View scan history

Use case	View scan history
Primary actor	Registered (verified) user
Goal in context	Review earlier predictions exactly as they were shown
Preconditions	The user is logged in
Trigger	The user opens the History page from the navigation bar
Scenario	1. The front end calls `GET /scans/history` with the session token. 2. The system returns the user's scans, newest first. 3. The History page shows a card for each scan (heatmap, modality, class, confidence and date); the Profile page shows the six most recent. 4. The user opens a card to see the full report.
Exceptions	Missing or expired token (401; the user is returned to the login page); no scans (empty state)
Priority	Medium
When available	First release
Frequency of use	Occasional
Channel to actor	Web browser
Secondary actors	None
Channels to secondary actors	–
Open issues	Individual scans cannot be deleted or exported

2.2.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system rather than specific behaviours. They are summarised in Table 2.6 and elaborated in the subsections that follow.

Table 2.6: Non-functional requirements

ID	Attribute	Requirement
NFR1	Performance	A prediction request, including the gates, classification, Grad-CAM and report, shall complete in about one to two seconds on a laptop-class machine.
NFR2	Correctness of served models	The served models shall be exactly the ones whose metrics are reported, verified by a reproduction check at start-up.
NFR3	Security	Passwords shall be stored hashed, sessions shall be signed and time-limited, and expensive or sensitive routes shall be rate limited.
NFR4	Usability	A first-time user shall be able to analyse a scan without instructions, and every rejection shall be explained in plain English.
NFR5	Safety and ethics	The system shall never present its output as a diagnosis, and shall display a non-diagnostic disclaimer with every result.
NFR6	Maintainability	Configuration and secrets shall be read from one environment file, and the back end shall be covered by an automated test suite.

2.2.4.1 Performance Requirements

The prediction path must run on a laptop without a data-centre GPU. The measured end-to-end latency on the development laptop was about 1.3 s per prediction (mean 1.26 s for X-ray and 1.29 s for CT over eight requests each, including report generation), and the status endpoints answer in a few milliseconds (Section 6.1.4). The server loads all six models once at start-up, so no model is loaded per request. Uploads are capped at 10 MB to bound memory use and processing time.

2.2.4.2 Reliability Requirements

The server shall not serve a model that does not match its data. At start-up, each checkpoint's claimed validation accuracy (or Dice and IoU for the segmentation model) is recomputed and compared against a strict tolerance, and the server refuses to start on a mismatch. This check needs the validation data on disk; the hosted container does not ship them, so there the check is switched off and it is run instead on the development machine. Concurrent requests shall not corrupt each other's Grad-CAM computation; access to each model is therefore serialised with a lock. Failures of external services (LLM, SMTP) shall degrade gracefully to fixed fallbacks rather than failing the request, and unexpected server errors shall return a generic message without internal details.

2.2.4.3 Security Requirements

Passwords are hashed with bcrypt and never stored in plain text. Sessions are HS256-signed JSON Web Tokens [22] valid for seven days. Email-verification links expire after 24 hours and password-reset links after one hour, and both are single-use. The forgot-password route returns the same response whether or not the email exists, so it cannot be used to discover accounts. Google OAuth uses a signed `state` parameter bound to a nonce cookie. Email links and OAuth redirects are built only from an allow-listed set of origins, which prevents open redirects. Per-client rate limits apply to the prediction, chat and authentication routes. A signed-in user whose email address is not verified receives HTTP 403 from the prediction and chat routes.

2.2.4.4 Usability Requirements

The interface follows a single linear flow: choose the scan type, upload, wait for the analysis, and read the result. Rejections and errors are shown as plain-English messages with *Try again* and *Choose another image* buttons. The result page shows the class, confidence, probability bars, heatmap and report on one screen. The interface supports light and dark themes and works on mobile browsers through the public link.

2.2.4.5 Safety, Ethics and Maintainability Requirements

Every report and chat reply is generated under a system prompt that forbids diagnosis, treatment advice and prognosis, and the same non-diagnostic disclaimer appears whether the report comes from the LLM or from the fallback template. The landing page states that the tool is a research prototype for decision support. For maintainability, all secrets and URLs are read from `backend/.env` through a single configuration module, model checkpoints record their own class names and backbone, and the back end has an automated suite of 81 tests.

2.3 Software and Hardware Requirement Specifications

Tables 2.7 and 2.8 list the software and hardware used to develop and run the system.

Table 2.7: Software requirements

Component	Software / version
Operating system	Windows 10/11, macOS or Linux
Programming languages	Python 3.12, JavaScript (ES2022)
Deep learning	PyTorch 2.11, torchvision 0.26, TorchXRayVision
Image processing	Pillow, OpenCV, scikit-image, NumPy, SciPy, ImageHash
Machine-learning utilities	scikit-learn, matplotlib
Back end	FastAPI 0.141, Uvicorn, Pydantic 2, python-multipart, slowapi
Authentication	bcrypt, PyJWT, Google OAuth 2.0
Database	SQLite 3 (Python standard library)
Generative AI	Groq API (`openai/gpt-oss-120b`)
Email	Gmail SMTP with an app password
Front end	React 19, React Router 7, Vite 8
Testing	pytest 9, httpx (FastAPI TestClient), Playwright
Design and modelling	PlantUML (UML use case, class, sequence, state and component diagrams)
Deployment	Docker and Docker Compose, Caddy (HTTPS), AWS EC2; ngrok for local demonstrations
Browser	Any modern browser (Chrome, Edge, Firefox, Safari)

Table 2.8: Hardware requirements

Item	Development / training	Serving (hosted deployment)
Processor	Intel Core i5/i7 or equivalent	2 vCPUs, x86-64 (AWS EC2 `t3.small`)
GPU	NVIDIA RTX 3050 Laptop GPU (4 GB) locally; Google Colab GPU for full training runs	Not required; inference runs on the CPU
Memory	16 GB RAM	2 GB RAM with a 4 GB swap file; the application uses about 1 GB
Storage	About 10 GB for datasets, checkpoints and environments	30 GB disk for the Docker image, checkpoints and build cache
Network	Internet access for dataset download and the LLM and email APIs	Public HTTPS (ports 80 and 443); outbound access to the LLM and email services
Client	–	Any device with a modern web browser

CHAPTER 3

PROPOSED SYSTEM

This chapter describes the proposed system at a functional level: what it does with an uploaded image, who it is meant for, the design principles behind it, its advantages and applications, and the boundary of what it does and does not attempt.

3.1 Description of the Proposed System

The proposed system is a web application that screens chest X-rays and chest CT slices for lung disease. Its central idea is that a medical image classifier should answer only when the input is something it was trained to handle, and that every answer should be explained. Figure 3.1 shows the framework, which has three layers.

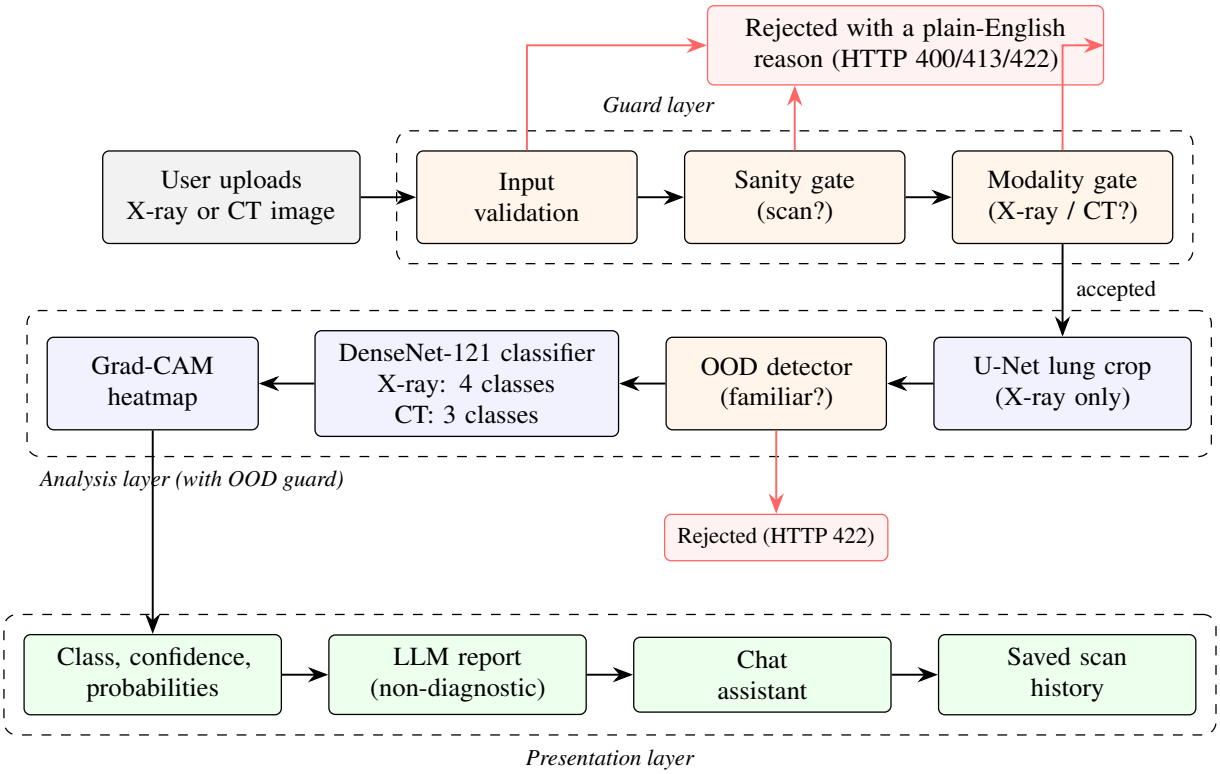


Figure 3.1: Framework of the proposed system

Guard layer. Before any disease model runs, the upload passes three checks in order.

- *Input validation* rejects files larger than 10 MB, files that are not decodable images, near-blank images and colour photographs (real X-rays and CT slices are greyscale).
- The *sanity gate*, a ResNet-18 binary classifier trained separately for each modality, decides whether the image is a medical scan at all or a photograph, document or screenshot.

- The *modality gate*, another ResNet-18, decides whether the image is an X-ray or a CT slice, and rejects it if it does not match the selected modality or if its confidence is below 0.90.

Each rejection returns a plain-English reason to the user.

Analysis layer. Accepted X-rays are cropped to the lung fields by a U-Net segmentation model, so that the classifier sees the same framing it was trained on and is not distracted by text, markers or borders outside the lungs. The *OOD detector* then measures the distance of the image's feature vector from the centre of the training features, and rejects the image if it lies farther away than 99% of the training images. Finally, the modality's DenseNet-121 classifier predicts the class probabilities, and Grad-CAM produces a heatmap of the regions that most influenced the prediction.

Presentation layer. The result (class, confidence and the full probability vector) is passed to an LLM that writes a short plain-language explanation under a strict non-diagnostic prompt; a fixed template is used if the LLM is unavailable. A chat assistant, scoped to questions about the tool and the result, answers follow-up questions. For signed-in users, the scan, heatmap and report are saved to their history.

3.2 Description of Target Users

The system is designed for the following users:

- **Clinicians and health workers in under-served settings**, such as primary health centres and small clinics with no on-site radiologist, who can use the tool to prioritise scans for expert review.
- **Radiologists**, as a second reader and triage aid that highlights the region the model attended to.
- **Medical students and educators**, who can use the heatmaps, probabilities and explanations to discuss how imaging findings relate to disease classes and how models can go wrong.
- **Researchers and developers** of medical AI, as a worked example of input guards, start-up verification and honest external evaluation.

In every case, the tool is a decision-support aid. It does not replace a qualified radiologist or physician, and the interface says so on every result.

3.3 Advantages and Applications of the Proposed System

3.3.1 Design Principles

The following design principles shaped the system and are the source of its advantages.

- **Refuse rather than guess.** A discriminative classifier has no "none of the above" option. The guard layer gives the system one, so a photograph or the wrong scan type is rejected with a reason instead of being forced into a disease class.
- **Explain every answer.** Grad-CAM heatmaps, the full probability vector, a low-confidence warning and a plain-language report make each prediction inspectable.
- **Serve what was measured.** Every checkpoint stores its validation metric; in strict mode the server recomputes that metric at start-up and refuses to start if it does not match, so the model being demonstrated is the model whose numbers are reported.
- **Transfer learning from relevant data.** The X-ray classifier starts from DenseNet-121 weights pre-trained on a large aggregate of chest radiographs, and the CT classifier from ImageNet weights. Both are fine-tuned with a class-weighted loss to handle severe class imbalance.
- **Safety by construction.** The report and chat prompts forbid diagnosis and treatment advice, and the same disclaimer appears whether the LLM call succeeds or falls back to the template.

3.3.2 Advantages

- A single application handles two imaging modalities and seven disease classes.
- Invalid inputs are caught before they can produce a misleading result.
- Results are explained visually and in plain language, which builds appropriate trust.
- The system runs on an ordinary laptop, with a GPU optional, and can be reached from any browser, including on a phone, through a public HTTPS link.
- User accounts and scan history allow results to be revisited exactly as they were first shown.

3.3.3 Applications

- Triage of chest X-rays for tuberculosis and pneumonia in high-volume or low-resource settings.
- A second-reader aid for CT slices.
- Teaching material on medical AI and explainability.

Each clinical application assumes that the models are first validated, and if necessary retrained, on images from the site where they would be used; Chapter 7 shows why this step cannot be skipped.

3.4 Scope (Boundary of the Proposed System)

3.4.1 In Scope

The system accepts a single 2D image per request: a frontal chest X-ray or an axial chest CT slice in PNG or JPEG format. It classifies X-rays as COVID-19, Normal, Pneumonia or Tuberculosis

and CT slices as Benign, Malignant or Normal, with a heatmap, a report and a chat assistant. It provides registration, email verification, Google sign-in, password reset and change, scan history, rate limiting and a public HTTPS deployment on a cloud server.

3.4.2 Out of Scope

The system does not process DICOM files or full 3D CT volumes . It does not localise or measure nodules, and it does not support multi-label findings, lateral X-rays or other body parts. It does not store patient identifiers. It is not a certified medical device and does not give a diagnosis, treatment advice or prognosis. Continuous integration, multi-server scaling and high availability were deliberately left out of this prototype.

3.4.3 Assumptions and Constraints

Assumptions. Uploaded scans are reasonably oriented and of diagnostic quality; one scan is analysed per request; and users have basic familiarity with a web browser.

Constraints. The training data is limited to public datasets (19,217 X-ray images and 1,036 usable CT images after cleaning); the CT dataset comes from a single institution, which limits generalisation; inference must be fast enough on a laptop without a dedicated server GPU; and the hosted deployment runs on a small CPU-only cloud instance (2 vCPUs, 2 GB RAM).

CHAPTER 4

SYSTEM DESIGN

This chapter describes the design of the system: its overall architecture and the prediction pipeline at its centre, the main classes of the back end, the sequence of interactions for an analysis request, the database schema, the states the user interface moves through, and the datasets the models were trained on.

4.1 Architecture of the System

The system follows a three-tier architecture, shown in Figure 4.1.

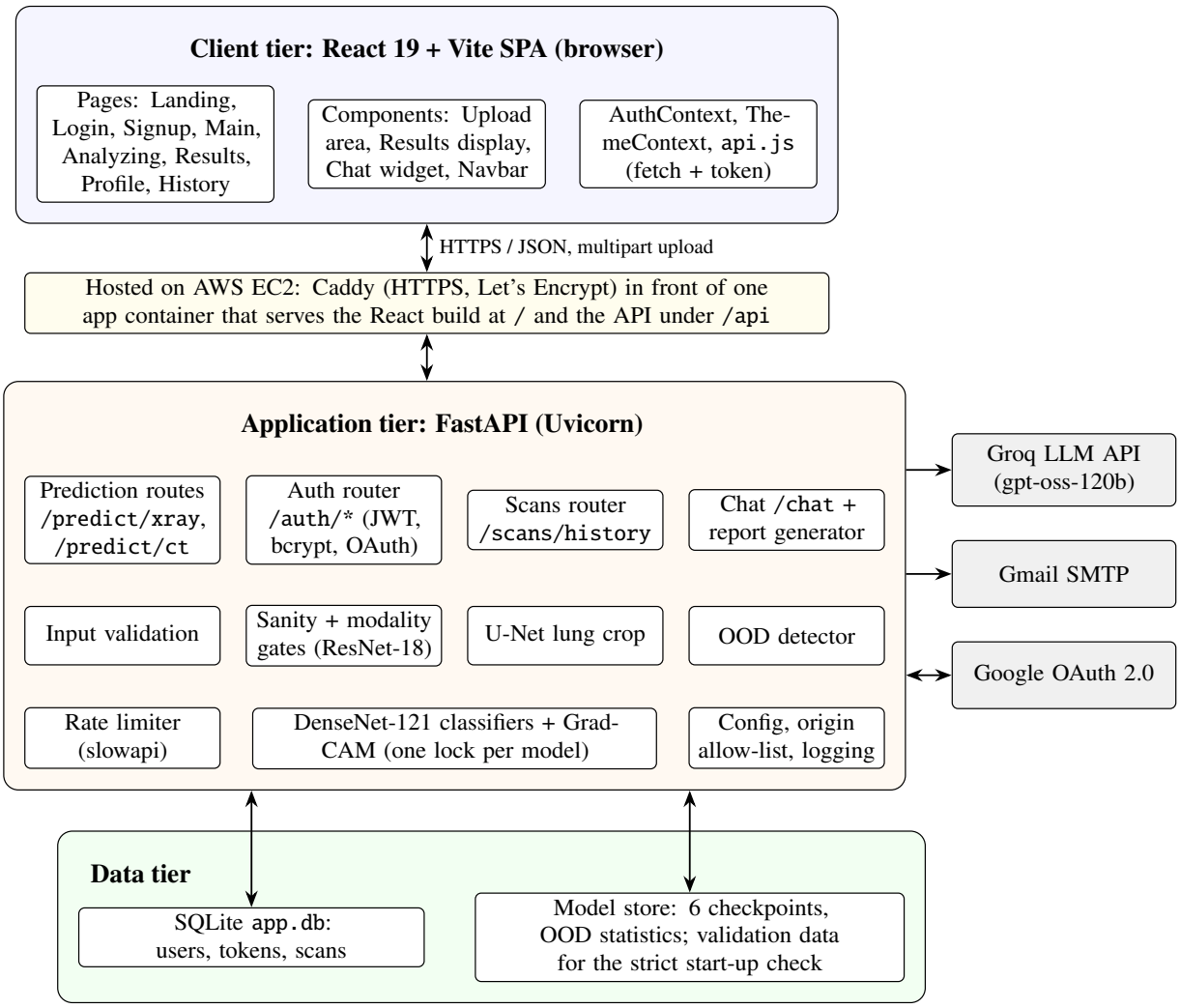


Figure 4.1: Three-tier architecture of the system

- **Client tier.** A React 19 single-page application built with Vite. Its pages implement the landing page, the authentication screens, the upload screen, the analysing screen, the results,

the profile and the history. Authentication and theme state are kept in React Context providers. Prediction, chat and history requests go through one module (`api.js`), and the authentication calls in the auth context share its base URL and error type; both attach the session token and turn every failure into a readable message.

- **Application tier.** A FastAPI application served by Uvicorn. It exposes fifteen routes: two prediction routes (`/predict/xray`, `/predict/ct`), nine authentication routes under `/auth`, `/chat`, `/scans/history`, and the status routes `/` and `/health`. At start-up it loads six models (two sanity gates, the modality gate, two disease classifiers and the U-Net) and two OOD statistics files and, in strict mode, verifies each model against its validation data.
- **Data tier.** A SQLite database with three tables (users, tokens, scans) and a model store holding the checkpoints, OOD statistics and validation data.
- **External services.** The Groq API generates reports and chat replies, Gmail SMTP sends verification and reset emails, and Google OAuth 2.0 provides sign-in.

In the hosted deployment (Section 5.3.12), one Docker container serves the built front end at `/` and the API under `/api` from a single origin, behind a Caddy reverse proxy that terminates HTTPS. Only Caddy is reachable from the internet; the application container is never exposed directly. For local demonstrations, an ngrok tunnel to `vite preview` gives the same single-origin layout.

Two separate prediction routes were chosen over a single route with a modality field. Each route has a fixed response schema with its own class list, and selecting the model becomes a routing decision rather than a branch on a user-supplied field; checking for a modality mismatch then reduces to asking whether the image is plausible for the model the route already commits to. Algorithm 1 gives the prediction pipeline that both routes share.

The order of the stages matters. The sanity gate runs first because the modality gate is a closed-set classifier with no "neither" option: in testing, a random photograph was scored above 92% "xray" by the modality gate and also passed the OOD check. The two gates see the uncropped upload; only images that pass both are lung-cropped. The OOD check and the classification with Grad-CAM run inside one continuous hold of the classifier's lock, because both use the same shared model object and Grad-CAM attaches hooks to it; holding the lock across both steps prevents an overlapping request from seeing another request's hooks.

Algorithm 1 Prediction pipeline (`_run_prediction`)

Input: upload f, route modality m (xray or ct), optional user u
Output: class, confidence, probabilities, heatmap, report, warnings; or an HTTP error

if u is signed in **and** not email-verified **then**
  return HTTP 403
end if
$img :=$ decode f // 413 if > 10 MB; 400 if empty or not an image
if std(grey(img)) < 10 **or** mean channel spread > 20 **then**
  return HTTP 400 // blank or colour photo
end if
$x := \text{Preprocess}_{\text{ImageNet}}(img)$ // greyscale to 3 channels, 224 by 224
if SanityGate$_m(x)$ is not scan **then**
  return HTTP 422 "not a scan"
end if
$(m_g, p) :=$ ModalityGate(x) // m_g: modality predicted by the gate
if $p < 0.90$ **then**
  return HTTP 422 "modality uncertain"
end if
if m_g is not m **then**
  return HTTP 422 "wrong modality"
end if
if $m =$ xray **then**
  $x := \text{Preprocess}_{\text{XRV}}(\text{LungCrop}(img))$
end if
acquire lock of classifier m
$d :=$ Euclidean distance from Classifier$_m$.features(x) to centre$_m$
if $d >$ threshold$_m$ **then**
  release lock; **return** HTTP 422 "out of distribution"
end if
$(probs, heatmap) :=$ Classifier$_m(x)$ with Grad-CAM
release lock
if $\max(probs) < c_m$ **then**
  add low-confidence warning // $c_{xray} = 0.50,\ c_{ct} = 0.55$
end if
$report :=$ LLMReport$(m, probs)$, or template on failure
if u is signed in **then**
  save scan for u
end if
return result

4.2 Selection of Design Alternatives

Design choices were made with a weighted decision matrix, a formal decision-making tool in which each alternative is rated against weighted criteria and the weighted sum decides. The criteria follow from the non-functional requirements and constraints: accuracy evidence on chest images (performance), parameter count as a proxy for CPU inference cost (performance and scalability on a laptop), the availability of chest-radiograph pre-trained weights (reliability of transfer learning), and library support with a clean last convolutional block for Grad-CAM

(maintainability). Table 4.1 applies the matrix to the choice of classifier backbone; each rating is on a scale of 1 to 5.

Table 4.1: Weighted decision matrix for the disease-classifier backbone

Backbone	Params	Accuracy on chest images (30%)	CPU cost (25%)	Chest-X-ray weights (25%)	Maintain-ability (20%)	Weighted score
VGG-16	138 M	3	1	1	4	2.20
ResNet-50	25.6 M	4	3	4	5	3.95
Inception-v3	23.8 M	4	3	1	3	2.80
EfficientNet-B0	5.3 M	4	5	1	4	3.50
DenseNet-121	**8.0 M**	**5**	**4**	**5**	**5**	**4.75**

DenseNet-121 scores highest: it is the architecture of CheXNet [1], TorchXRayVision [4] provides it pre-trained on chest radiographs (only ResNet-50 among the others has such weights), and at about 8 million parameters it runs comfortably on a laptop CPU. Table 4.2 records the other architectural decisions and the quality attribute each one serves.

Table 4.2: Other design decisions and the criterion behind each

Decision	Alternatives	Criterion	Choice and justification
Back-end framework	Flask, Django, FastAPI	Maintainability	FastAPI: request and response validation with Pydantic and automatic OpenAPI documentation
Database	PostgreSQL, MongoDB, SQLite	Reliability	SQLite: three small tables on one host; no database server to run or fail
Input gates	DenseNet-121, ResNet-18	Performance	ResNet-18: a binary task needs only a small model, so the gates add negligible latency
Prediction routes	One route with a modality field, one route per modality	Scalability	One route per modality: a fixed response schema each, and a new modality is a new route rather than a new branch

4.3 Class Diagram

The back end is written mostly as modules of functions, with a small number of classes holding the loaded models and their state. Figure 4.2 shows these classes and their relationships.

- **PredictionService** is a modelling abstraction for the prediction logic, which the code implements as module-level functions in `main.py`. It owns five **ModelBundle**s (two sanity gates, the modality gate and two classifiers), two **OODDetector**s and one **SegBundle**, and it builds the **PredictionResponse** returned to the client.
- **ModelBundle** wraps one loaded network together with its class names, backbone name, device, the validation accuracy stored in the checkpoint, the accuracy reproduced at start-up, and a lock that serialises access to the model.

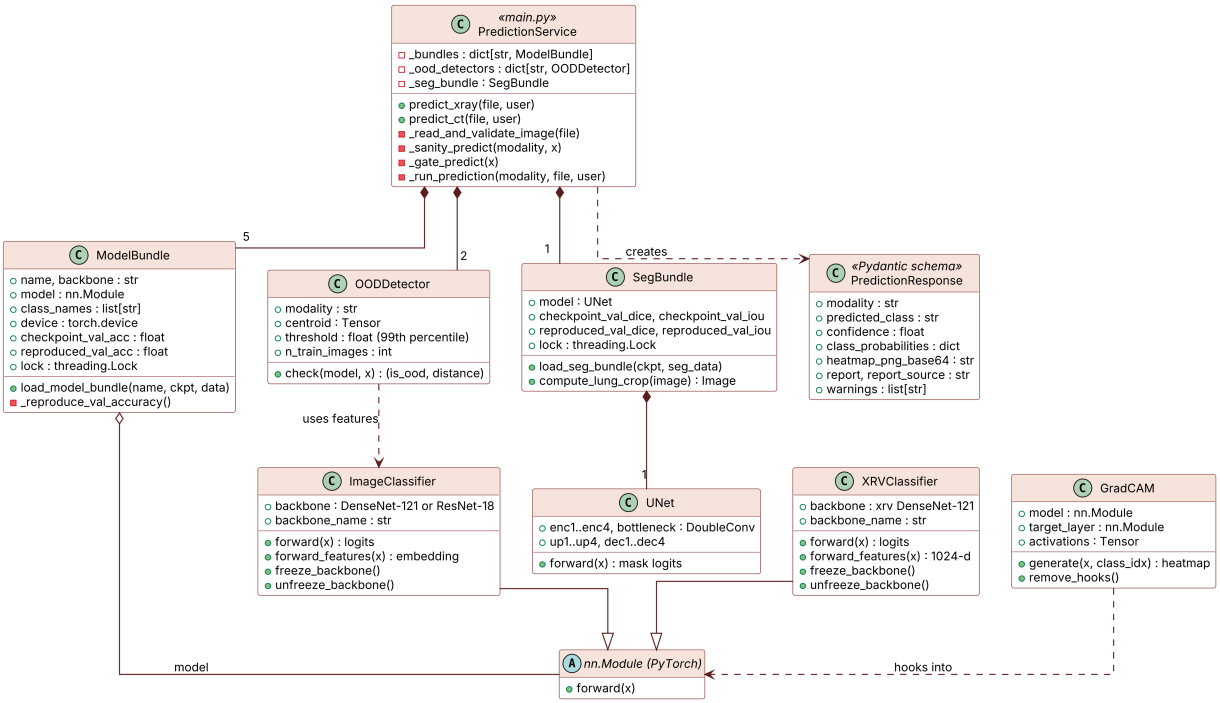


Figure 4.2: Class diagram of the main back-end classes (PlantUML)

- **ImageClassifier** and **XRVClassifier** are PyTorch modules. `ImageClassifier` wraps a torchvision DenseNet-121 or ResNet-18 with a new head; `XRVClassifier` wraps the TorchXRayVision DenseNet-121 with a new four-class head. Both expose `forward_features()`, which the OOD detector uses, and methods to freeze and unfreeze the backbone during training.
- **OODDetector** holds the training-feature centroid and the 99th-percentile distance threshold for one modality.
- **SegBundle** wraps the **UNet** segmentation model with its claimed and reproduced Dice and IoU, and provides `compute_lung_crop()`.
- **GradCAM** registers a forward hook on the classifier's last convolutional block and produces the class-activation heatmap; a new instance is created for each request.

4.4 Design Patterns

Seven design patterns are applied across the client and the server. Figure 4.3 shows where each sits on the path of a request, and Table 4.3 names the code that implements it and the benefit it brings.

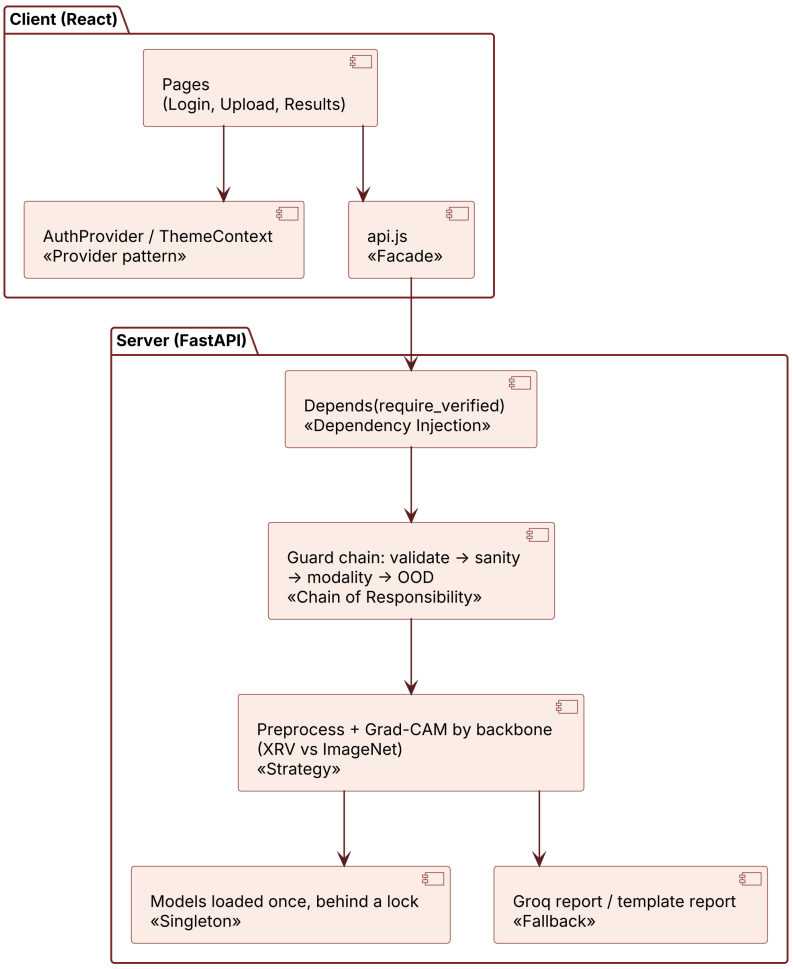


Figure 4.3: Design patterns on the path of a prediction request (PlantUML component diagram)

Table 4.3: Design patterns applied in the system

Pattern	Where in the code	Benefit
Facade	`frontend/src/api.js`	Pages never call `fetch` directly; one module owns the base URL, the session token and error handling
Provider (React Context)	`AuthProvider`, `ThemeContext`	Login and theme state are shared with every page without passing props down
Dependency Injection	FastAPI `Depends(require_verified_if_logged_in)`	One verified-user check is reused by every protected route
Chain of Responsibility	`_run_prediction`: validation, sanity gate, modality gate, OOD	Each guard can stop the request with its own reason; a new guard is one more link
Strategy	Preprocessing and Grad-CAM chosen by `is_xrv_backbone`	X-ray and CT preprocessing are swapped without changing the pipeline
Singleton	`_ensure_models_loaded` with a lock	The six models are loaded once and shared safely, not reloaded per request
Fallback	`report.generate_report`	A report with the disclaimer is always returned, even when the LLM is unavailable

4.5 Sequence Diagram

Figure 4.4 shows the sequence of messages when a signed-in user analyses a chest X-ray. The React UI sends the image as a multipart request with the session token. The back end validates the image and calls, in turn, the sanity gate, the modality gate, the U-Net and the classifier (twice: once for the OOD features and once for the prediction with Grad-CAM), then the LLM, and finally stores the scan. If any check fails, the request ends at that stage and the UI shows the reason with a *Try again* button. For a CT scan the sequence is the same except that the lung-crop step is skipped.

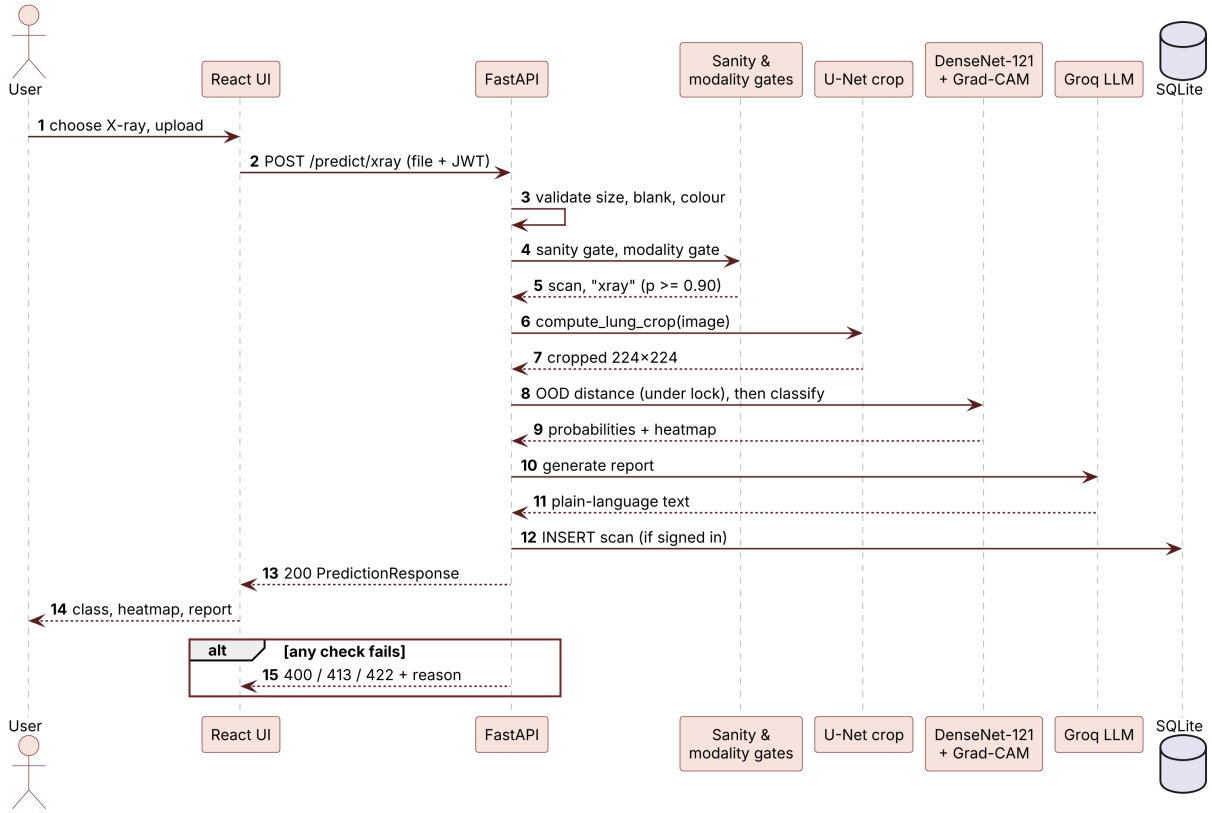


Figure 4.4: Sequence diagram for analysing a chest X-ray (PlantUML)

4.6 ER Diagram and Schema

Persistence uses plain SQLite through Python's standard `sqlite3` module, without an ORM; the schema is small and stable and does not justify a database server. Figure 4.5 shows the entities. A user has zero or more tokens (email-verification and password-reset tokens) and owns zero or more scans. Foreign keys are enforced.

Table 4.4 describes the `scans` table. Every authenticated prediction is stored with its full probability vector, heatmap and report, so the history page reproduces exactly what the user saw at prediction time.

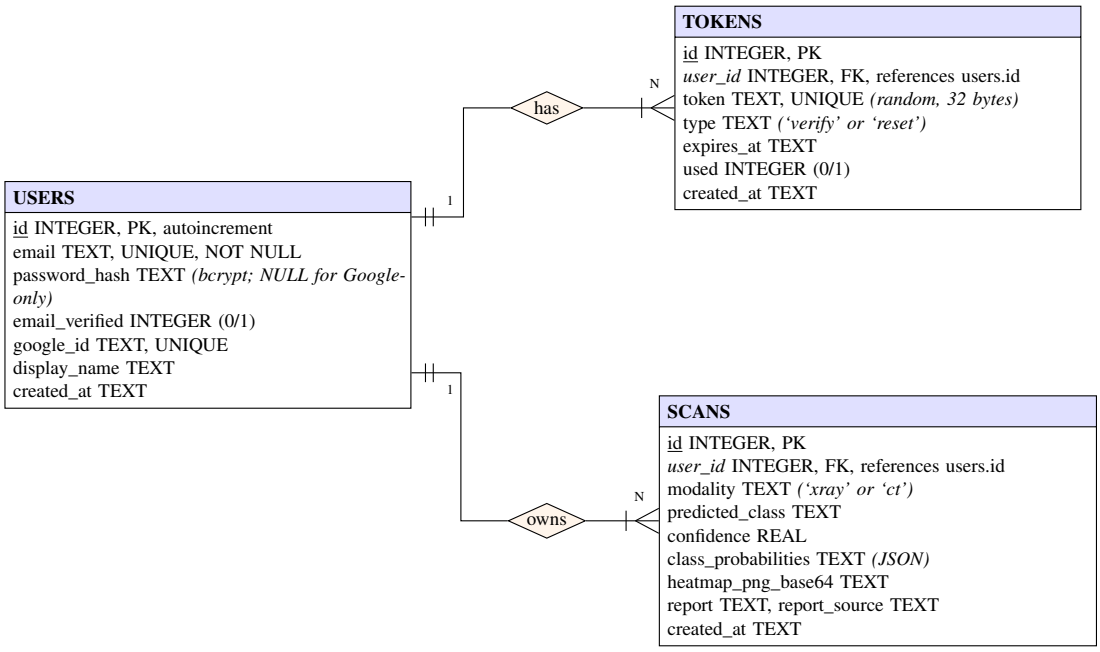


Figure 4.5: ER diagram of the database

Table 4.4: Schema of the `scans` table

Column	Type	Purpose
`id`	INTEGER PK	Unique scan identifier
`user_id`	INTEGER FK	Owner of the scan (`users.id`)
`modality`	TEXT	`xray` or `ct`
`predicted_class`	TEXT	Top-1 predicted label
`confidence`	REAL	Top-1 softmax probability
`class_probabilities`	TEXT (JSON)	Full probability vector
`heatmap_png_base64`	TEXT	Grad-CAM overlay as a base64 PNG data URI
`report`	TEXT	Plain-language report
`report_source`	TEXT	`groq` or `template_fallback`
`created_at`	TEXT	Creation timestamp

4.7 State Transition Diagram

Figure 4.6 shows the states of the user interface. A signed-out visitor sees the landing page. After signing up the user is signed in but unverified, and the application shows only a "verify your email" screen until the link is clicked; Google sign-in produces a verified account directly. A verified user chooses a modality, selects a file and starts the analysis. The analysis ends either in a result, from which the user can open the chat or start again, or in a rejection or error, from which the user can retry. A 401 response at any point (log out or expired session) returns the user to the signed-out state. The front end mirrors the back end's rules: every protected route applies the same verified-user check that the back end enforces with HTTP 403.

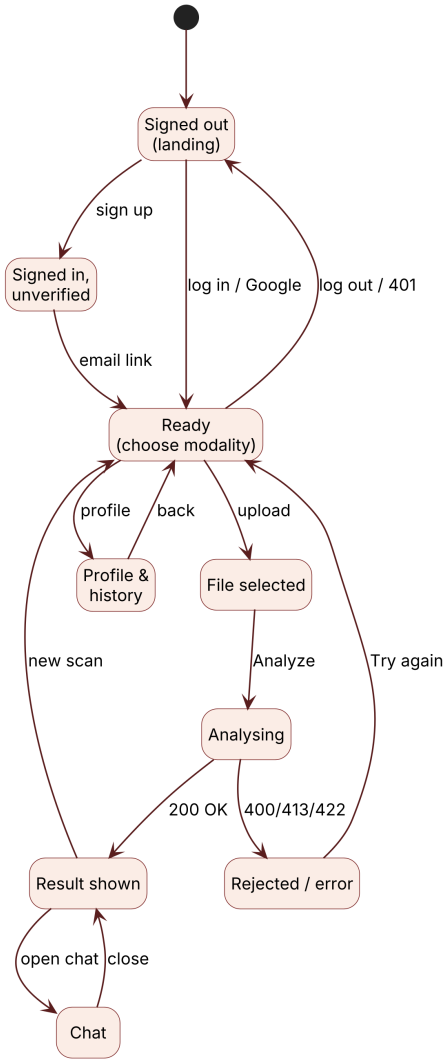


Figure 4.6: State transition diagram of the user interface (PlantUML)

4.8 Dataset Description

Three public datasets were used for the disease classifiers, plus two auxiliary datasets for the segmentation model and the gates. Table 4.5 summarises them.

The two X-ray sources were merged into four classes, with exact duplicates removed by SHA-256 content hashing (59 duplicates in total, almost all within a single source). The CT dataset was cleaned and resplit after the leakage audit described in Section 5.3.1. Table 4.6 gives the final splits, and Figure 4.7 shows the class distribution. Both datasets are heavily imbalanced: Normal outnumbers Tuberculosis by about 22 times in the X-ray data, and Malignant outnumbers Benign by about 4.2 times in the CT data. Inverse-frequency class weights in the loss function counter this during training.

The sources behind each X-ray class matter for what the classifier can learn. Within the COVID-19 Radiography Database [23], the Viral Pneumonia images are taken from the Kaggle *Chest X-Ray Images (Pneumonia)* collection, which consists of paediatric radiographs (patients aged one to five years) from a single hospital in Guangzhou [25]; 1,341 of the database's Normal images

Table 4.5: Datasets used

Dataset	Type	Classes used	Raw size and notes
COVID-19 Radiography Database [23]	X-ray	COVID-19, Normal, Pneumonia (viral)	3,616 COVID, 10,192 Normal, 1,345 Viral Pneumonia, with per-image lung masks; Lung Opacity (6,012) excluded
Tuberculosis Chest X-ray Database [24]	X-ray	Normal, Tuberculosis	3,500 Normal, 700 TB; no lung masks
IQ-OTH/NCCD Lung Cancer Dataset [18]	CT	Benign, Malignant, Normal	1,097 slices from 110 patients, single institution, flat per-class JPEGs (no patient IDs)
Lung-mask dataset (derived)	X-ray	Lung field mask	21,165 image/mask pairs from the COVID-19 Radiography Database; split 16,933 / 2,116 / 2,116
Non-scan negatives (curated)	Mixed	Photo, document, screenshot	7,460 negatives for the X-ray sanity gate and 3,000 for the CT sanity gate

come from the same collection, while the other X-ray sources consist mainly of adult radiographs. Every Pneumonia image in this project is therefore paediatric.

Table 4.6: Final train / validation / test splits (random seed 42)

Modality	Class	Train	Val	Test	Total
X-ray	COVID-19	2,856	357	357	3,570
	Normal	10,949	1,369	1,368	13,686
	Pneumonia	1,070	134	134	1,338
	Tuberculosis	498	62	63	623
	Total	*15,373*	*1,922*	*1,922*	*19,217*
CT	Benign	96	12	12	120
	Malignant	400	50	50	500
	Normal	333	42	41	416
	Total	*829*	*104*	*103*	*1,036*

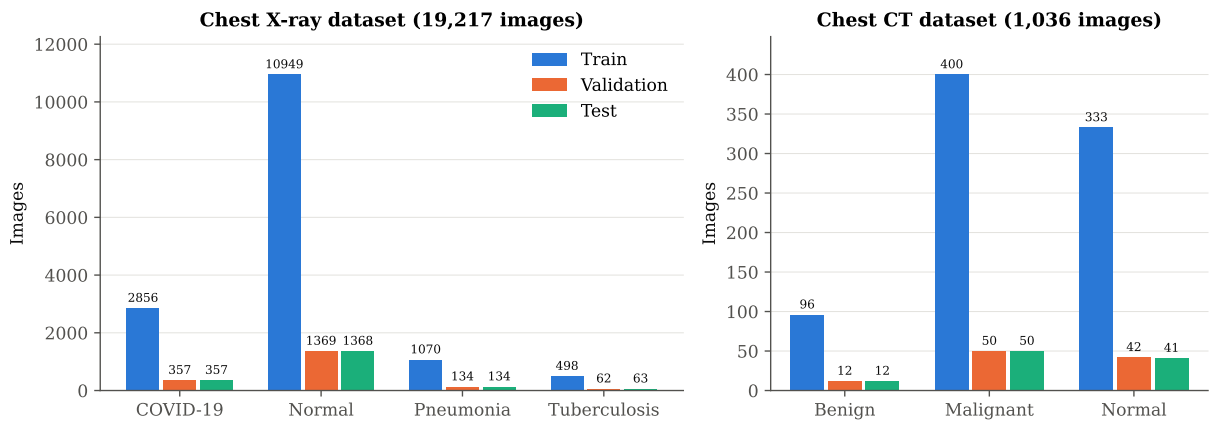


Figure 4.7: Class distribution of the final X-ray and CT datasets by split

Figure 4.8 shows sample image and mask pairs from the lung-segmentation dataset, which was used to train the U-Net that crops X-rays to the lung fields.

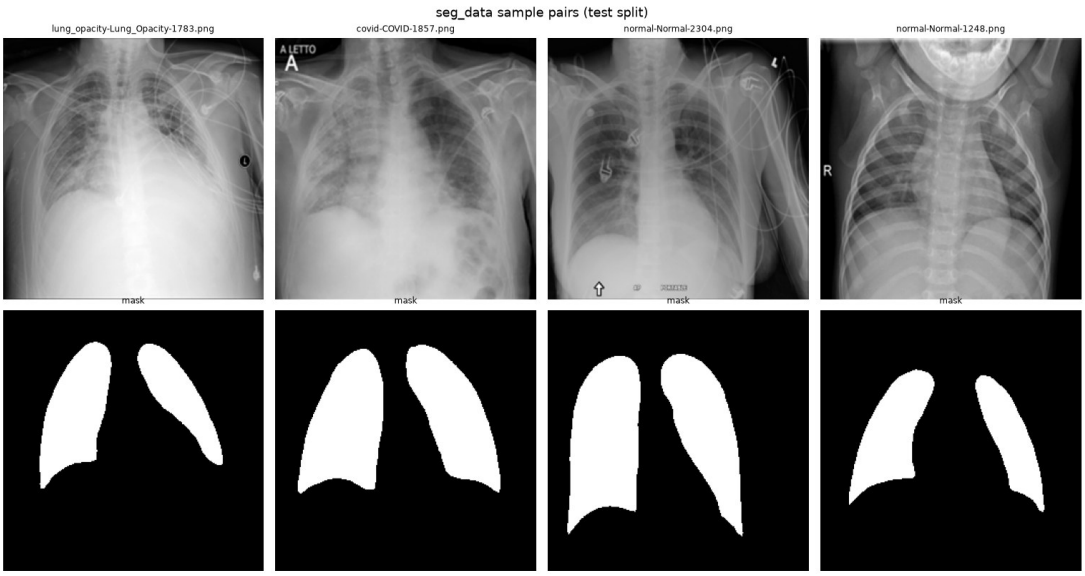


Figure 4.8: Sample image / lung-mask pairs from the segmentation dataset (test split)

CHAPTER 5

IMPLEMENTATION

This chapter explains the methodology followed in the project and then describes each module of the system: its purpose, inputs and outputs, and the algorithm or code structure behind it. The implementation is in Python (PyTorch and FastAPI) for the models and back end, and JavaScript (React) for the front end.

5.1 Proposed Methodology

Figure 5.1 shows the methodology. It is a pipeline from data to deployment with two feedback loops, both of which were exercised during the project.

1. **Data acquisition.** The public datasets in Table 4.5 were downloaded, together with the lung masks and a curated pool of non-scan images.
2. **Cleaning and audits.** Exact duplicates were removed, burned-in artifacts in the tuberculosis images were cleaned, and the CT dataset was checked for near-duplicate slices across splits (Section 5.3.1).
3. **Preprocessing.** X-rays were cropped to the lung fields and every image was resized to 224 by 224 with the normalisation expected by its backbone.
4. **Model training.** Each classifier was trained by transfer learning: two epochs with the backbone frozen, then full fine-tuning with a lower learning rate and an inverse-frequency weighted loss.
5. **Offline evaluation.** Each model was evaluated on its held-out test split. An implausible result sends the work back to step 2: the first CT model scored 100% on its test split, which turned out to be leakage between splits.
6. **Integration and serving.** The models were integrated into the FastAPI back end behind the guard layer, with a start-up reproduction check, and connected to the React front end.
7. **Live-serving audit.** The same test images were sent through the real HTTP routes. A mismatch between the training and serving crops was found and fixed in the preprocessing step (step 3).
8. **External validation.** The deployed system was tested on images from independent sources to measure how far the results generalise.

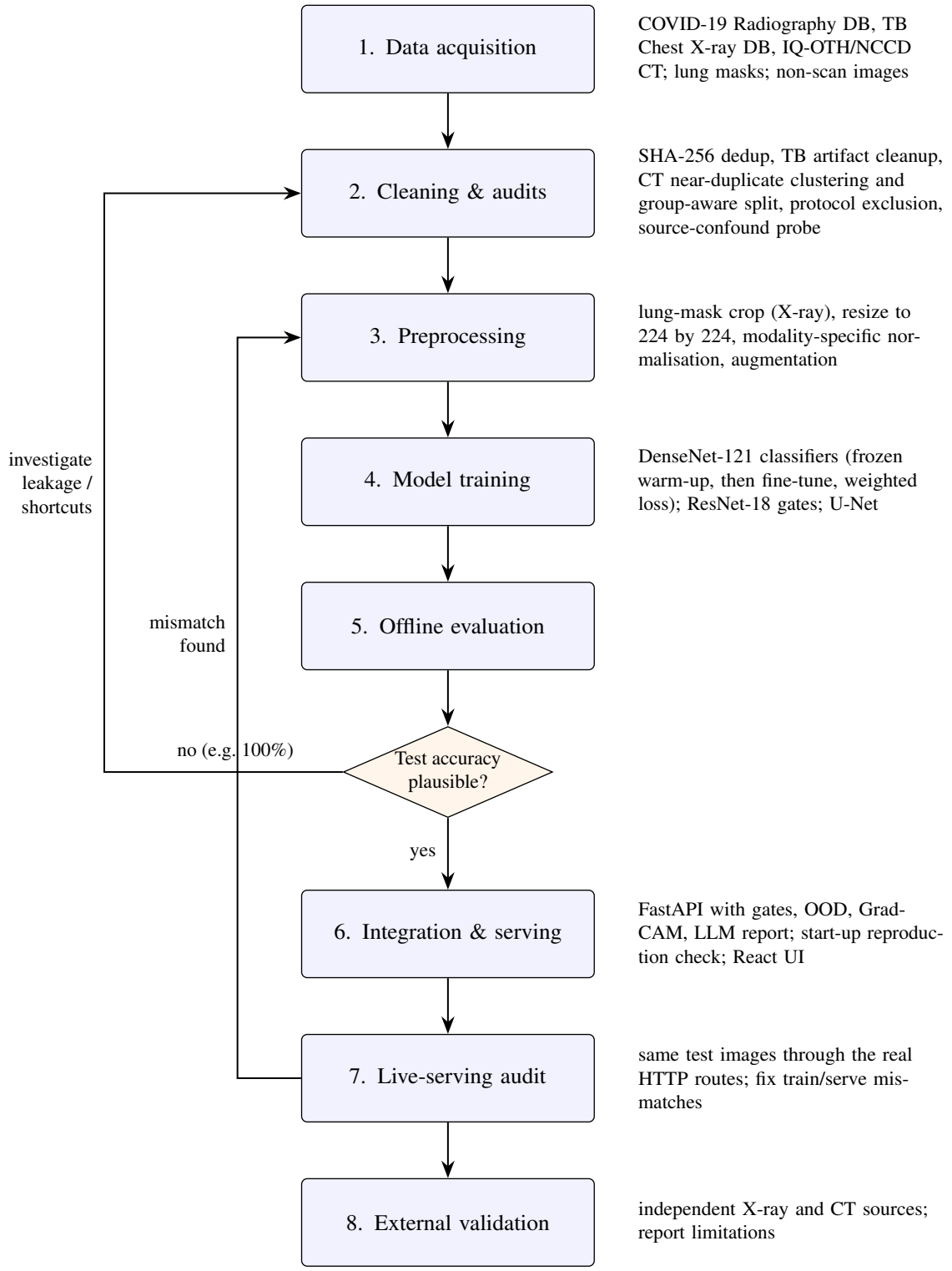


Figure 5.1: Methodology followed in the project

5.2 Algorithm Selection for Each Module

For every module the standard existing algorithms were identified and one was selected against the constraints of Section 3.4: inference on a laptop CPU, a small single-institution CT dataset, and an explanation for every prediction. Table 5.1 summarises the alternatives and the reason for each choice; the selected algorithms are described module by module in the next section.

Table 5.1: Existing algorithms identified and the algorithm selected for each module

Module	Existing algorithms identified	Selected	Reason
Disease classifier	VGG-16, ResNet-50, Inception-v3, EfficientNet-B0, DenseNet-121	DenseNet-121	Highest weighted score (Table 4.1); chest-radiograph weights; 8.0 M parameters
Lung segmentation	Otsu thresholding, FCN, SegNet, U-Net	U-Net [6]	Skip connections preserve lung borders and suit small biomedical datasets; validation Dice 0.98
Scan and modality gates	Rule-based checks only, OOD check only, CNN binary classifier	ResNet-18 + rule checks	The OOD check alone let a real CT slice through the X-ray route
OOD detection	Maximum softmax probability, ODIN, Mahalanobis distance, energy score, feature-centroid distance	Centroid distance, 99th percentile	Needs no retraining; softmax confidence is weak for unfamiliar input [19]; a Mahalanobis covariance in 1,024 dimensions cannot be estimated from 829 CT images
Explainability	Saliency maps, LIME, SHAP, Grad-CAM	Grad-CAM [7]	Class-specific and needs one backward pass; LIME and SHAP need hundreds of forward passes per image
Class imbalance	Oversampling, focal loss, class-weighted cross-entropy	Weighted cross-entropy	Handles a 3% Tuberculosis class without duplicating images
CT leakage audit	Random split, patient-ID split, perceptual hash with SSIM	Perceptual hash + SSIM, union-find	The dataset has no patient IDs; the random split gave a false 100% test accuracy

5.3 Description of Modules

The modules are described below in the order in which a request passes through them, preceded by the offline modules that prepared the data and the models.

5.3.1 Module 1: Data Preparation and Audit

Scripts: `prepare_xray.py`, `tb_artifact_detect.py`, `tb_clean.py`, `ct_leakage_audit.py`, `ct_resplit_grouped.py`, `ct_table_shortcut_audit.py`, `xray_source_audit.py`
Input: raw datasets
Output: cleaned, deduplicated, split image folders `data/xray` and `data/ct_scan`

Four audits changed the data on which the system is trained and evaluated (Algorithm 2).

Tuberculosis artifacts. The audit found that 53.1% of Tuberculosis images carried a colour tint and/or a burned-in redaction box, against only 0.37% of Normal images from the same source (Figure 5.2). Such a class-correlated artifact is a ready-made shortcut: a model can learn "tinted image implies TB". Both classes from that source were cleaned the same way: of the 700 TB

Algorithm 2 Data-quality audit pipeline (offline, one-time)

TB burned-in artifact audit (X-ray):
  scan all 4,200 TB-source images for colour tint (HSV saturation) and box-shaped near-white regions (redactions)
  desaturate tinted images; inpaint small boxes ($< 5\%$ of frame); exclude images with large boxes
Patient-leakage audit (CT):
  compute a 64-bit perceptual hash of all 1,097 CT images; flag pairs with small Hamming distance
  confirm each pair by SSIM of at least 0.90; group confirmed pairs into clusters with union-find
  resplit train/val/test (80/10/10) treating each cluster as one indivisible group
Protocol-mismatch audit (CT):
  detect scanner-table visibility per image and cross-check with image resolution
  exclude the non-standard-protocol subset (61 images)
Source-confound audit (X-ray):
  within the Normal class, train a logistic-regression probe on 32 by 32 thumbnails to predict the source database

images, 328 were left unchanged, 298 were desaturated and 146 were inpainted (some images needed both), and 72 were excluded, leaving 628 (623 after deduplication).

CT leakage. The first CT split assigned images to splits at random, and the first CT model scored 100% on its test set. The leakage audit found 111 clusters of near-duplicate slices (287 confirmed pairs, using perceptual hashing and SSIM [26]); 47 clusters (177 images) spanned more than one split, and 35 of the 109 original test images had a near-duplicate in the training set. Because the Kaggle export contains no patient IDs, clusters of adjacent slices served as a proxy for patients. The data were resplit so that each cluster lies wholly in one split, and a re-audit found zero cross-split leaks. A separate audit found 61 Malignant images from a different acquisition protocol (no visible scanner table, non-standard image resolution) that the model could separate on protocol rather than pathology; these were excluded, leaving 1,036 images.

Source confound. In the X-ray data, COVID-19 images come only from one database and Tuberculosis only from another, so source and diagnosis are confounded. Within the Normal class, which draws from both databases, a deliberately weak logistic-regression probe on 32 by 32 thumbnails recovered the source database with 97.1% accuracy (AUC 0.996) on 3,495 images per source.

5.3.2 Module 2: Disease Classifier Training

Scripts: `ml/train_xray_txrv.py` (X-ray), `ml/train.py` with `ml/model.py` and `ml/dataset.py` (CT)
Input: cleaned image folders
Output: checkpoints storing weights, class names, backbone name and best validation accuracy

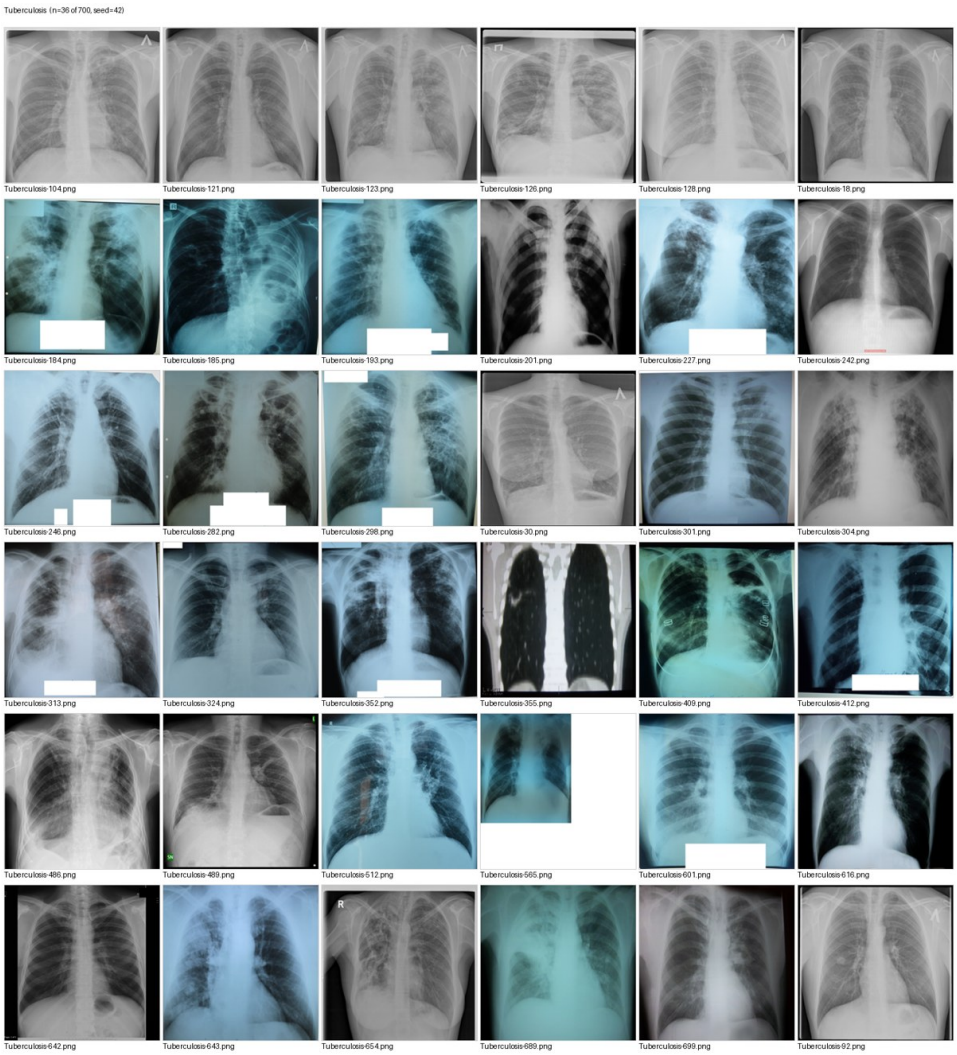


Figure 5.2: Random sample of raw Tuberculosis images before cleaning, showing colour tints and white redaction boxes

Both classifiers use DenseNet-121 [3], whose dense connections reuse low-level texture features across layers, which helps when fine-tuning on datasets far smaller than ImageNet, while its roughly 8 million parameters keep CPU inference practical. The X-ray classifier uses the TorchXRayVision `densenet121-res224-all` weights [4], pre-trained on a large aggregate of chest radiographs, with its 18-pathology head replaced by a new 4-class head and the library's own preprocessing (single channel, intensities scaled to roughly -1024 to 1024). This replaced an earlier ImageNet-pretrained X-ray model. The CT classifier uses ImageNet weights [5] with a new 3-class head and ImageNet normalisation. Training follows Algorithm 3 with the settings in Table 5.2.

5.3.3 Module 3: Sanity Gate and Modality Gate

Functions: `_sanity_predict(modality, x)`, `_gate_predict(x)`
Input: the uncropped upload as a 224 by 224, ImageNet-normalised tensor
Output: "scan" / "not_scan"; "xray" / "ct" with confidence

Algorithm 3 Two-phase transfer-learning training

Input: training set D_{tr}, validation set D_{val}, pretrained backbone, E epochs, W warm-up epochs
Output: best checkpoint by validation accuracy

compute class weights $w_c = N/(K n_c)$ // inverse frequency
replace classifier head with a K-way linear layer
for $e = 1$ to E **do**
    if e is at most W **then**
        freeze backbone; optimiser := Adam(head, learning rate 0.001)
    else if $e = W + 1$ **then**
        unfreeze backbone; optimiser := Adam(all, learning rate 0.0001)
    end if
    for each mini-batch (x, y) of augmented images **do**
        $loss$:= weighted cross-entropy$(f(x), y; w)$; back-propagate; update
    end for
    evaluate accuracy on D_{val}; if best so far, save checkpoint
end for

Table 5.2: Training configuration of the disease classifiers

Setting	X-ray classifier	CT classifier
Backbone	DenseNet-121 (TorchXRayVision weights)	DenseNet-121 (ImageNet weights)
Input	1 channel, 224 by 224, XRV scaling	3 channels, 224 by 224, ImageNet mean/std
Classes	4	3
Epochs (warm-up)	15 (2 frozen)	15 (2 frozen)
Batch size	32	32
Optimiser, learning rate	Adam [27]; 0.001 warm-up, 0.0001 fine-tune	Adam; 0.001 warm-up, 0.0001 fine-tune
Loss	Inverse-frequency weighted cross-entropy	Weighted cross-entropy (weights 2.88 / 0.69 / 0.83)
Augmentation	Horizontal flip ($p = 0.5$), rotation up to 10 degrees either way, brightness/contrast up to 0.1 either way	Same
Best validation accuracy	96.46% (epoch 12)	99.04% (epoch 6)
Random seed	42	42
Hardware	Google Colab GPU; RTX 3050 laptop for smoke tests	Same

Both gates are ResNet-18 [28] binary classifiers, small enough that their cost is negligible next to the classifier they protect. Two sanity gates are trained, one per modality, on real scans as positives against curated photographs, documents and screenshots as negatives (X-ray: 15,373 positives vs. 7,460 negatives; CT: 829 vs. 3,000). They reach 99.96% and 100% validation accuracy. The modality gate is trained on the X-ray and CT training splits relabelled by modality (15,373 X-ray and 829 CT images) and reaches 100% validation accuracy, as the two modalities look very

different. A request is rejected if the modality gate's confidence is below 0.90 or if it predicts the other modality. The modality gate was added because the OOD detector alone did not catch a real CT scan sent to the X-ray route: in feature space, a CT slice is still a real medical image, just the wrong kind.

5.3.4 Module 4: Lung Segmentation and Crop

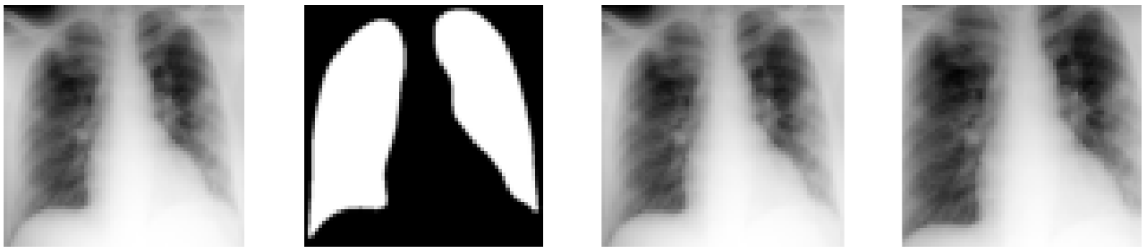
Function: `compute_lung_crop(image, seg_bundle)` in `seg_utils.py`
Input: an accepted X-ray image
Output: a 224 by 224 image cropped to the lung fields

The classifier was trained on lung-cropped images, so serving raw uploads to it changes the input distribution; in testing, a genuine COVID-19 X-ray was classified as Tuberculosis at 98.6% confidence because of this mismatch. The segmentation model is a 4-level U-Net [6] with 32 base channels and skip connections, producing a single-channel mask. Its checkpoint was trained outside this repository on the lung-mask dataset; its architecture was reconstructed from the checkpoint and is confirmed by the strict start-up check, which reproduces its validation Dice (0.983 against 0.989 claimed). Algorithm 4 gives the crop procedure and Figure 5.3 shows it on a real image.

Algorithm 4 U-Net lung crop

Input: X-ray image I, U-Net g
Output: cropped 224 by 224 image

1 $M := [\text{sigmoid}(g(\text{resize}_{256}(I))) > 0.5]$ // binary lung mask
2 $B :=$ bounding box of M with 4-pixel padding, scaled to the size of I
3 **if** M is empty **or** $\text{area}(B)/\text{area}(I)$ is outside $[0.05, 0.98]$ **then**
4 $\quad I_c :=$ crop 8% from every border of I // fallback, as used for TB training data
5 **else**
6 $\quad I_c :=$ crop I to B
7 **end if**
8 **return** I_c resized to 224 by 224 // squash-resize, exactly as in training



(a) raw input (b) U-Net mask (c) seg. crop (d) border crop

Figure 5.3: Lung-crop pipeline on a real X-ray: (a) input, (b) U-Net mask, (c) segmentation crop, (d) border-crop fallback

The final squash-resize in line 8 was added after the live-serving audit (Section 7.4): earlier, the crop returned a non-square box, which the X-ray preprocessing then centre-cropped to a square, removing lung margins that the training images had kept.

5.3.5 Module 5: Out-of-Distribution Detector

Class: `OODDetector` (`ood_detector.py`); thresholds computed by `scripts/compute_ood_thresholds.py`
Input: the preprocessed tensor and the modality's classifier
Output: accept or reject, and the distance

A softmax classifier has no built-in way to say "none of the above", and its confidence is a weak signal for unfamiliar input [19]. The detector instead works in the classifier's own penultimate-layer feature space (after global average pooling, before the head). Offline, every training image x_i of the modality is embedded as a feature vector z_i, and the centroid c and threshold t are stored, where dist is the Euclidean distance:

$$c = \frac{1}{N}\sum_{i=1}^{N} z_i, \qquad t = Q_{0.99}\left(\{\mathrm{dist}(z_i, c)\}_{i=1}^{N}\right) \tag{5.1}$$

At inference an input x with feature vector z is rejected if $\mathrm{dist}(z, c) > t$, that is, if it lies farther from the centre of the training data than 99% of the training images do. The thresholds are $t = 8.72$ for X-ray (15,373 images, recomputed after the backbone change, since a new backbone defines a new feature space) and $t = 25.62$ for CT (829 images).

5.3.6 Module 6: Classification and Grad-CAM

Functions: `predict_with_heatmap()`, `predict_with_heatmap_xrv()` in `gradcam_utils.py`; `GradCAM` in `ml/gradcam.py`
Input: preprocessed tensor
Output: class index, probability vector, heatmap as a base64 PNG data URI

The classifier's softmax gives the probability vector. Grad-CAM [7] then explains the predicted class c: with A^k the k-th feature map of the last convolutional block, y^c the score for class c, and G_{ij}^{kc} the gradient of y^c with respect to A_{ij}^k,

$$w_k^c = \frac{1}{Z}\sum_i\sum_j G_{ij}^{kc}, \qquad L^c_{\text{Grad-CAM}} = \mathrm{ReLU}\left(\sum_k w_k^c A^k\right) \tag{5.2}$$

The map is upsampled to 224 by 224, normalised to $[0, 1]$, coloured with a jet colour map and alpha-blended (0.45) over the de-normalised input. A new `GradCAM` instance, with its own hook, is created for each request and its hook is removed afterwards. If the top probability is below 0.50 (X-ray) or 0.55 (CT), a low-confidence warning is attached to the response.

5.3.7 Module 7: Report Generation and Chat Assistant

Files: `report.py`, `chat.py`, `groq_client.py`
Input: modality, predicted class, confidence and probabilities (and, for chat, the message and recent history)
Output: report or reply text, and its source (`groq` or `template_fallback`)

The report prompt asks the Groq-hosted `openai/gpt-oss-120b` model for a two-to-three-sentence explanation for a non-medical reader and forbids diagnosis, treatment suggestions and speculation beyond the numbers. A fixed disclaimer is appended. If no API key is set or the call fails, a template report carrying the same disclaimer is used, so the disclaimer is never lost. (The originally planned Llama 3.3 70B model was retired from Groq during development.)

The chat assistant uses the same provider with a system prompt that restricts it to the tool, its output and its limitations; it must decline off-topic requests (code, trivia, creative writing) and any medical advice, treatment or prognosis, and direct the user to a qualified physician. It receives the current prediction as context and the last ten turns of conversation. Requests are capped at 2,000 characters per message and 50 history items.

5.3.8 Module 8: Start-up Reproduction Check

Functions: `load_model_bundle()` in `models.py`, `load_seg_bundle()` in `seg_utils.py`
Input: checkpoint files and validation data
Output: loaded, verified model bundles, or a refusal to start

Algorithm 5 Start-up reproduction check

for each checkpoint k in {xray, ct, gate, xray_sanity, ct_sanity, segmentation} **do**
    load weights, class names, backbone and claimed metric a_k from the checkpoint
    recompute the metric r_k on the validation split with the same preprocessing
    if r_k and a_k differ by more than tol_k **then** // tol: 0.001 accuracy, 0.01 Dice, 0.02 IoU
        log a critical error and **abort start-up**
    end if
end for
load OOD statistics; log configuration; start serving

This check exists because a mismatch between a checkpoint and the data on disk was caught once during the project, where a model had been trained on a different version of the data than the one present locally. Its cost is that start-up takes one to two minutes and needs the validation data on disk. The hosted container does not ship the validation data, so it sets `SKIP_REPRO_CHECK=1` and serves the checkpoints without re-scoring them; the strict check is run on the development machine instead (Table 6.4).

5.3.9 Module 9: Authentication, Sessions and Access Control

Files: `auth_routes.py`, `auth_utils.py`, `email_utils.py`, `public_origin.py`
Input: credentials, tokens, OAuth callbacks
Output: JWT session tokens, verified accounts, emails

Passwords (minimum eight characters) are hashed with bcrypt. Sessions are stateless HS256 JSON Web Tokens [22] valid for seven days. Verification and reset tokens are 32-byte random URL-safe strings stored in the `tokens` table with an expiry and a used flag. Emails are sent over Gmail SMTP with an HTML template, or printed to the server console when SMTP is not configured. Google sign-in uses the authorisation-code flow, with a signed `state` parameter bound to an HTTP-only nonce cookie to prevent cross-site request forgery. Links in emails and OAuth redirects follow the origin the user is actually on (local or public) but only if that origin is on an allow-list. Table 5.3 lists the API.

Table 5.3: REST API of the back end

Method	Route	Purpose	Limit
POST	`/predict/xray`, `/predict/ct`	Analyse an uploaded scan	20/min
POST	`/chat`	Chat assistant reply	20/min
POST	`/auth/signup`	Register; send verification email	30/hour
POST	`/auth/login`	Email/password login; returns JWT	20/min
GET	`/auth/me`	Current user	–
POST	`/auth/change-password`	Change password (signed in)	–
GET	`/auth/verify/{token}`	Verify email, redirect to app	–
POST	`/auth/forgot-password`	Send reset link (generic reply)	5/hour
POST	`/auth/reset-password/{token}`	Set a new password	–
GET	`/auth/google/login`, `/auth/google/callback`	Google OAuth flow	–
GET	`/scans/history`	Signed-in user's scans	–
GET	`/`, `/health`	Service status, loaded models	–

5.3.10 Module 10: Rate Limiting and Error Handling

File: `rate_limit.py`; exception handler in `main.py`
Input: every incoming request
Output: the request is passed on, or answered with HTTP 429 (rate limit) or a sanitised HTTP 500

Rate limits are applied per client IP with `slowapi`. Behind a reverse proxy every request reaches the back end from the proxy, so all visitors would share one IP; the limiter therefore reads the `X-Forwarded-For` chain, but only when the direct peer is a configured trusted proxy (the local Vite proxy in development, the Caddy container in the hosted deployment), scanning right to left

and skipping trusted hops so that a client cannot spoof its address. A global exception handler logs unexpected errors and returns a generic message without internal details.

5.3.11 Module 11: Front End

Files: `frontend/src` (pages, components, contexts, `api.js`)
Input: user actions in the browser
Output: rendered pages and calls to the REST API

The React application separates public routes (landing, login, signup, forgot and reset password, OAuth callback, email verified) from protected routes (main app, analysing, results, profile, history). The root route shows the landing page to signed-out visitors, a "verify your email" screen to unverified users, and the main application only to verified users; every protected route applies the same check. The upload screen offers a choice of modality and a drag-and-drop area. The analysing screen sends the request while showing a checklist of stages (checking the scan type, running the classifier, drawing the heatmap, writing the explanation); because the back end returns a single response, the first three stages advance on a timer and the last completes only when the real result arrives. The screen then moves to the results page or to a retryable error state, which is also used for every 422 rejection. The results view shows the class, confidence, probability bars and the report, and toggles between the heatmap and the original image. A floating chat widget offers quick-reply chips. The History page lists every saved scan, and the Profile page shows the account and the six most recent scans. Theme and session are stored in `localStorage`.

5.3.12 Module 12: Deployment

Files: `Dockerfile`, `docker-compose.yml`, `Caddyfile`, `backend/serve.py`, `scripts/deploy_ec2.sh`, `DEPLOYMENT_EC2.md`
Input: the committed code, the six checkpoints and a server-side environment file holding the secrets
Output: the application at a public HTTPS address

The application is hosted on a single AWS EC2 instance (`t3.small`: 2 vCPUs and 2 GB RAM with a 4 GB swap file, Ubuntu 24.04) using Docker Compose with two containers.

- **Application container.** A multi-stage `Dockerfile` first builds the React front end, then installs the CPU-only build of PyTorch and the Python dependencies and copies in only the runtime files: the back end, the model code, the six checkpoints and the built front end. The image runs as an unprivileged user. Inside it, `backend/serve.py` serves the front end at `/` and the FastAPI application under `/api` from a single origin, so the browser never makes cross-origin calls, and client-side routes such as `/reset-password/...` fall back to `index.html`. Every model is loaded once while the image is built, which checks that the checkpoints load and caches the pre-trained TorchXRayVision weights, so the container starts without network downloads. Because the validation data are not in the image, the start-up reproduction check is switched off here (Module 8).

- **Caddy container.** Caddy terminates HTTPS with an automatically renewed Let's Encrypt certificate for a free `sslip.io` domain, rejects request bodies larger than 12 MB, and forwards requests to the application container, which is not published on the host. Rate limiting trusts `X-Forwarded-For` only from Caddy's fixed address.

The SQLite database and the logs live on named Docker volumes, so user accounts and scan history survive rebuilds and redeployments. A deploy script uploads the committed code and weights, rebuilds the image, restarts the services and waits until `/api/health` answers. The public address works from any device, including phones, without the development machine being switched on. For local demonstrations, a launch script can instead build the front end, serve it with `vite preview` (which proxies `/api` to the back end on the loopback interface) and expose it through an ngrok HTTPS tunnel.

CHAPTER 6

TESTING

Testing in this project covers two different questions. The first is the usual software question: does each route, check and screen behave as specified? The second is specific to machine-learning systems: do the models deliver, through the deployed system, the accuracy we claim? This chapter describes the test plan and the test cases for both; the model-accuracy results themselves are presented in Chapter 7.

6.1 Test Plan and Test Cases

Table 6.1 summarises the types of testing carried out.

Table 6.1: Test plan

Type	What is tested and how	Tools
Unit tests	Pure functions in isolation: client-IP extraction from forwarded headers, origin resolution and allow-listing, OAuth state signing and tampering	pytest
Integration (API) tests	Each route through FastAPI's TestClient, with the real start-up (all six models loaded, and verified when the validation data are present), a throw-away database, and SMTP and Groq mocked	pytest, httpx
Guardrail (negative) tests	Wrong-modality scans, photographs, screenshots, documents, blank, noise, corrupt and empty files sent to the live routes	Python harness over HTTP
Concurrency test	Twelve simultaneous predictions compared with the same requests run sequentially	Python threads
Model verification	Start-up reproduction check of all six checkpoints at every server start in strict mode	built into the back end
Model evaluation	Offline test split, the same test split through the live API, and external images (Chapter 7)	evaluation scripts
Performance test	Latency of prediction, chat and status routes	Python timing harness
Acceptance tests	End-to-end user scenarios in the browser against the requirements of Chapter 2	Manual, Chrome and mobile browser

6.1.1 Unit and Integration Test Plan and Test Cases

The back-end suite in `backend/tests` contains 81 automated tests in seven files. A session-scoped fixture starts the real application with all six models, so when the validation data are present every test run also exercises the start-up reproduction check; a temporary SQLite database

is used, and the SMTP and Groq clients are replaced by mocks so that no email or API call leaves the machine. The suite passes in full (81 passed in the final verification run). Table 6.2 lists representative test cases.

Table 6.2: Representative unit and integration test cases

ID	Test case	Input	Expected result	Status
T01	Predict a real X-ray	Test-split chest X-ray to `/predict/xray`	200; 4 class probabilities summing to 1; heatmap data URI	Pass
T02	Predict a real CT slice	Test-split CT slice to `/predict/ct`	200; 3 class probabilities; heatmap	Pass
T03	History saved for signed-in user	X-ray with a valid token	Scan appears in `/scans/history`	Pass
T04	No history for anonymous call	X-ray without a token	History of other users unchanged	Pass
T05	Wrong modality	X-ray sent to `/predict/ct`	422 with a modality message	Pass
T06	Blank image	Solid grey PNG	400 “appears to be blank”	Pass
T07	Colour photograph	Colourful synthetic image	400 “colour content”	Pass
T08	Not an image	Text file renamed `.png`	400 “not a valid image”	Pass
T09	Empty file	0-byte upload	400 “file is empty”	Pass
T10	Uncertain modality	Gate confidence mocked below 0.90	422 “modality uncertain”	Pass
T11	Crop skipped on rejection	Wrong modality, crop function spied	Lung crop never called	Pass
T12	Sign up and fetch profile	New email, valid password	200 with token; `/auth/me` returns user	Pass
T13	Duplicate email	Existing email	409	Pass
T14	Short password / bad email	7-character password; malformed email	422	Pass

ID	Test case	Input	Expected result	Status
T15	Login	Correct, wrong and unknown credentials	200 / 401 / 401	Pass
T16	Protected route without token	`/auth/me` with no or garbage token	401	Pass
T17	Change password	Correct and wrong current password	200 and re-login works / 401	Pass
T18	Client IP behind proxy	Forwarded-For chains from trusted (Vite, Caddy) and untrusted peers	Real IP used only from a trusted proxy; spoofed left entries ignored	Pass
T19	Chat with LLM unavailable	Message with Groq failing	200 with `template_fallback`	Pass
T20	Chat input limits	Empty, >2,000 chars, >50 history items	400 / 422 / 422	Pass
T21	Chat rate limit	21 messages in a minute from one IP	21st returns 429	Pass
T22	Links follow origin	Signup / reset from local and public origins	Links use the caller's allow-listed origin	Pass
T23	Non-allow-listed origin	`?origin=` set to an unknown site	Falls back to the configured front end	Pass
T24	OAuth state protection	Forged, missing and tampered `state`	Redirect to login with an error; no session	Pass
T25	Health	`GET /health`	200, all five classifier/gate models listed	Pass
T26	LLM reply cut off	Groq reply truncated at the token limit	Report falls back to the template	Pass
T27	Single-origin hosting (`serve.py`)	`/`, client routes, assets and `/api/health`	SPA fallback to `index.html`; API at `/api`; missing asset 404	Pass

6.1.2 Guardrail Test Cases

A separate harness sent prepared inputs to the running server over HTTP. In the first row, three CT slices were sent to the X-ray route and three X-rays to the CT route; in the next four rows, each of three files was sent to both routes. The last two rows used the X-ray route only. Table 6.3 shows the outcome for each category.

Table 6.3: Guardrail test cases (live HTTP)

Input	Count	Observed response	Status
CT slices sent to the X-ray route, and X-rays sent to the CT route	6	422, "does not look like a chest X-ray / CT"	Pass
Colour photographs (landscape, object) and an application screenshot	6	400, colour content	Pass
Greyscale photograph, scanned document, random noise	6	422, "does not look like a chest X-ray / CT" (sanity gate)	Pass
Solid grey, solid blue, near-blank image	6	400, blank / no content	Pass
Corrupt (truncated) file, non-image, empty file	6	400, invalid or empty	Pass
Oddly framed X-rays (zoomed, very wide, very tall, mostly blank)	4	3 classified, 1 rejected (mostly blank); no crash	Pass
Lung Opacity X-rays (a real class not in the label set)	10	200; classified as COVID-19 (9) or TB (1)	Known limitation

The last row is a deliberate negative test. Lung Opacity images are genuine chest X-rays from the same source database, so they pass every gate and the OOD check, and the classifier assigns them to one of its four classes. The guard layer protects against wrong *kinds* of input, not against diseases the model was never trained on; this is recorded as a limitation.

6.1.3 Concurrency and Model-Verification Tests

Because FastAPI runs synchronous route handlers in a thread pool and Grad-CAM attaches hooks to a shared model, overlapping requests were a real risk; an early version failed with an error when one request's hook was still attached while another ran. After the per-model lock was introduced, twelve predictions (six X-ray and six CT) were sent simultaneously and compared with the same requests sent one after another. All twelve succeeded, every concurrent result matched its sequential result exactly (zero mismatches), and all twelve heatmaps were distinct. In strict mode, the start-up reproduction check (Table 6.4) acts as a regression test for every served model each time the server starts.

Table 6.4: Start-up reproduction check (live start-up)

Model	Claimed in checkpoint	Reproduced	Passed
X-ray classifier	acc 0.9646	acc 0.9646	Yes
CT classifier	acc 0.9904	acc 0.9904	Yes
Modality gate	acc 1.0000	acc 1.0000	Yes
X-ray sanity gate	acc 0.9996	acc 0.9996	Yes
CT sanity gate	acc 1.0000	acc 1.0000	Yes
Segmentation U-Net	Dice 0.9890, IoU 0.9786	Dice 0.9830, IoU 0.9671	Yes

6.1.4 Performance Testing

Latency was measured on the development laptop by timing sequential requests to the running back end (Table 6.5). A prediction, including the gates, the OOD check, classification, Grad-CAM and LLM report generation, took about 1.3 s on average; status routes answered in a few milliseconds. These are small samples meant to show the order of magnitude, not a load test. Requests to one modality are serialised by the model lock, so throughput under heavy concurrent load was not measured and is listed as future work.

Table 6.5: Measured latency (seconds)

Route	Requests	Mean	Median	Min	Max
`POST /predict/xray`	8	1.262	1.288	1.008	1.451
`POST /predict/ct`	8	1.285	1.373	0.563	1.674
`POST /chat`	3	0.661	0.366	0.313	1.305
`GET /health`	3	0.002	0.002	0.001	0.003

6.1.5 Acceptance Test Plan and Test Cases

Acceptance tests were carried out manually in the browser against the functional requirements, on the local deployment and through the public link on a mobile phone (Table 6.6).

Table 6.6: Acceptance test cases

ID	Scenario	Expected outcome	FR	Status
A01	New user signs up, receives email, clicks the link	Account verified; “Email verified” page; app unlocked	FR10	Pass
A02	Signed-in but unverified user opens the app	Only the “verify your email” screen is shown	FR10	Pass
A03	User signs in with Google	Returned to the app, signed in and verified	FR10	Pass

ID	Scenario	Expected outcome	FR	Status
A04	User requests a password reset and sets a new password	Reset email received; new password works; link cannot be reused	FR10	Pass
A05	User uploads a chest X-ray and clicks *Analyse*	Stage checklist, then class, confidence, probability bars, heatmap and report	FR1, FR5–FR8	Pass
A06	User uploads a CT slice on the CT tab	CT result with heatmap and report	FR1, FR5, FR7, FR8	Pass
A07	User toggles between heatmap and original	Image switches without a new request	FR7	Pass
A08	User uploads a CT slice on the X-ray tab	Clear rejection message with *Try again*	FR3	Pass
A09	User uploads a photograph or screenshot	Clear rejection message with *Try again*	FR2, FR3	Pass
A10	User asks the chat "What does the heatmap show?"	Plain-language explanation grounded in the current result	FR9	Pass
A11	User asks the chat for treatment advice or an unrelated task	Assistant declines and redirects	FR9	Pass
A12	User opens the History page after several analyses	History cards with the saved heatmap, class, confidence and date	FR11	Pass
A13	Back end stopped while the app is open	"Can't reach the analysis server" message, no crash	–	Pass
A14	App opened on a phone at the public HTTPS address	Full flow works over HTTPS	–	Pass
A15	Server started with a mismatched checkpoint	Server refuses to start with a clear log message	FR12	Pass

6.1.6 Limitations of the Testing

The automated suite covers the back end only; the front end has no automated component tests, and earlier Playwright scripts became out of date after the landing-page redesign. Token expiry and reuse for verification and reset links, account linking with Google, a real OOD rejection, the

413 size limit, and isolation of scan history between users are covered by manual testing but not yet by automated tests. The tests need the full checkpoints (and, for the strict check, the validation data), so they run on the development machine rather than in continuous integration. Because the hosted container skips the reproduction check, a checkpoint that no longer matched its data would be served there until the next strict run on the development machine.

CHAPTER 7

RESULTS & DISCUSSIONS

This chapter presents the results in four steps, each closer to real use than the last: the classifiers on their held-out test splits, the effect of the data audits, the deployed system on live requests, and the deployed system on images from independent sources. It then shows the working application and discusses what the results do and do not show. The training behaviour of the models is summarised first.

7.1 Training Behaviour

Figure 7.1 shows the training curves of the CT classifier. Loss falls sharply once the backbone is unfrozen after the two warm-up epochs, and validation accuracy peaks at 99.04% in epoch 6, which is the checkpoint kept; the spike in validation loss at epoch 11 shows why the best epoch is selected on validation data rather than taking the last one. Figure 7.2 shows the modality gate converging to 100% validation accuracy within two epochs, as expected for two visually very different modalities.

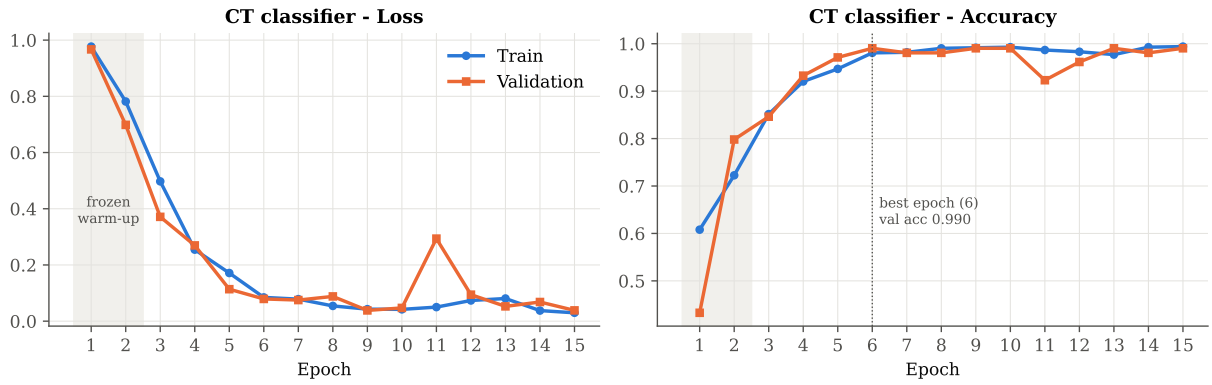


Figure 7.1: Training and validation loss and accuracy of the CT classifier (shaded: frozen-backbone warm-up)

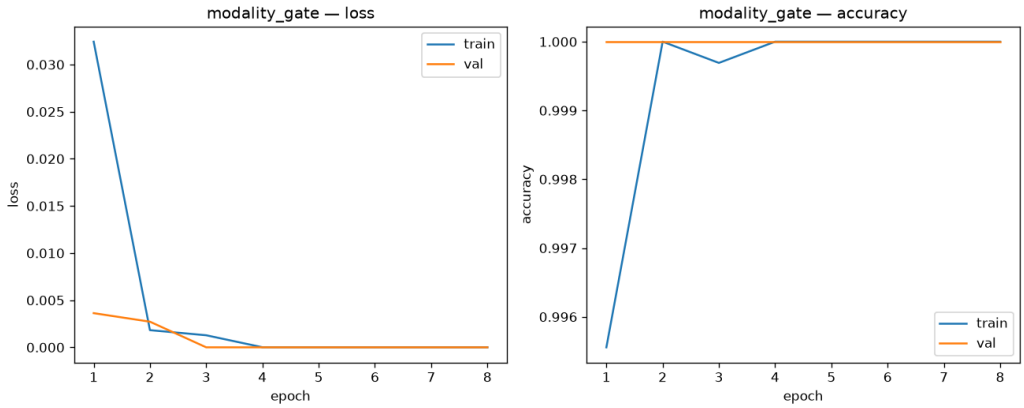


Figure 7.2: Training curves of the modality gate (ResNet-18)

7.2 Classification Performance on Held-out Test Data

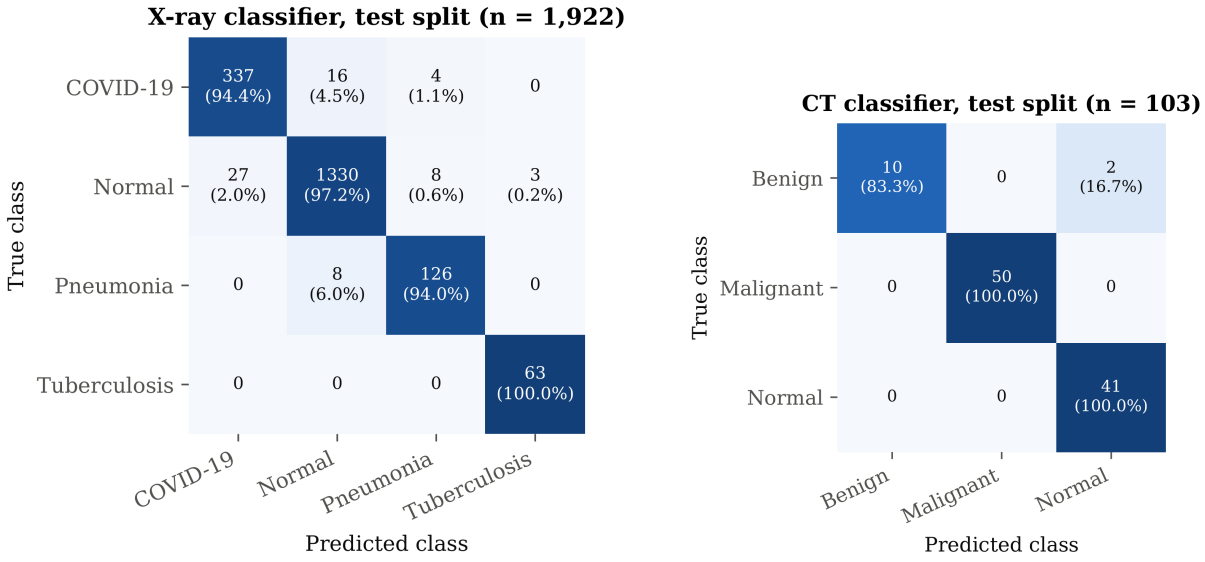
Tables 7.1 and 7.2 give the per-class precision, recall and F1-score of the two classifiers on their held-out test splits, which were never used for training or model selection. Because the X-ray test set is dominated by Normal images (1,368 of 1,922), macro-averaged scores are reported alongside overall accuracy. These figures measure the classifier alone: the test images were already lung-cropped when the dataset was built (Section 5.3.1), and they are fed to the classifier directly, without the additional serving-time steps (guards and U-Net crop) that Section 7.4 evaluates.

Table 7.1: X-ray classifier on the test split ($n = 1{,}922$; accuracy 96.57%)

Class	Precision	Recall	F1-score	Support
COVID-19	0.926	0.944	0.935	357
Normal	0.982	0.972	0.977	1,368
Pneumonia	0.913	0.940	0.927	134
Tuberculosis	0.955	1.000	0.977	63
Macro average	0.944	0.964	0.954	1,922
Weighted average	0.966	0.966	0.966	1,922

Table 7.2: CT classifier on the test split ($n = 103$; accuracy 98.06%)

Class	Precision	Recall	F1-score	Support
Benign	1.000	0.833	0.909	12
Malignant	1.000	1.000	1.000	50
Normal	0.953	1.000	0.976	41
Macro average	0.984	0.944	0.962	103
Weighted average	0.981	0.981	0.980	103



(a) X-ray classifier **(b)** CT classifier

Figure 7.3: Confusion matrices on the held-out test splits (counts and row percentages)

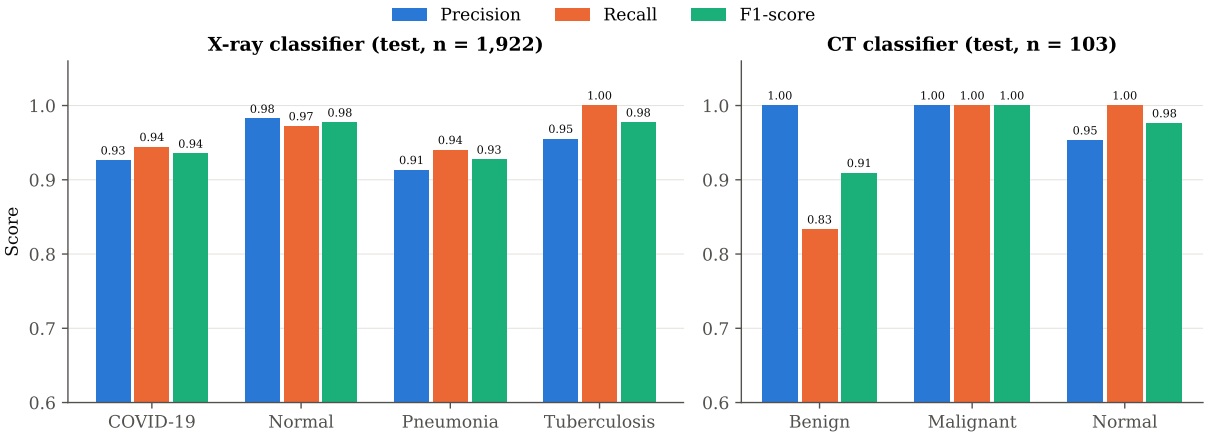


Figure 7.4: Per-class precision, recall and F1-score on the test splits

Both classifiers generalise consistently from validation to test (X-ray: 96.46% validation, 96.57% test; CT: 99.04% validation, 98.06% test). On X-ray, the most common errors are Normal predicted as COVID-19 (27 images) and COVID-19 predicted as Normal (16); all 63 Tuberculosis test images are recalled. On CT, the only errors are two Benign slices predicted as Normal; Benign is also the smallest class (12 test images), so each image is worth 8.3 points of Benign recall. All of these are *in-distribution* results: every test image comes from the same databases as the training images. The following sections test what that means in practice.

7.3 Effect of the Data Audits

The data audits changed the reported numbers, and in every case made them more trustworthy. The first CT model, trained on a random per-image split, scored 100% on its test set; the leakage audit showed that 35 of the 109 test images had a near-duplicate slice in the training set. After the group-aware resplit and the exclusion of 61 off-protocol images, test accuracy is 98.06%, with

two genuine errors. In the X-ray data, cleaning the burned-in tints and redaction boxes from the Tuberculosis images removed a feature that was present in 53.1% of TB images but only 0.37% of Normal images, and which a model could otherwise have used in place of the disease. Our figures are therefore lower than some published results on the same datasets (for example 99.54% on IQ-OTH/NCCD [17]), but they are not inflated by leakage.

7.4 Live-Serving Accuracy and the Crop Fix

The offline figures describe the classifier on pre-processed test tensors, not what a user receives from the deployed route. A dedicated audit sent the test images through the real `/predict/xray` route and found that live accuracy was 12 to 19 points below the offline figure, with errors drifting towards COVID-19 regardless of the true class: on a class-balanced sample, COVID-19 was predicted for 80% of served images (350 of 436), where about a quarter would be expected.

The cause was the lung-crop function. It could return a non-square bounding box, which the X-ray preprocessing then centre-cropped to its shorter side, silently removing lung margins that the training images had kept, since every training image had been squash-resized to a square. The fix made every return path of `compute_lung_crop()` end in the same squash-resize used to build the training data. Table 7.3 and Figure 7.5 show the effect.

Table 7.3: X-ray accuracy before and after the crop fix

Measurement (240-image class-balanced samples)	Offline	Before fix	After fix
Original (uncropped) uploads, serving path	96.57%[a]	88.33%	95.00%
Pre-cropped test images, serving path (simulated)	96.67%	80.00%	92.08%
Pre-cropped test images, live HTTP (233 of 240 served)	–	–	91.85%
Served images predicted as COVID-19	–	80%[b]	27%

[a] Reference figure for the full test split. [b] 350 of 436 served images in the earlier live audit.

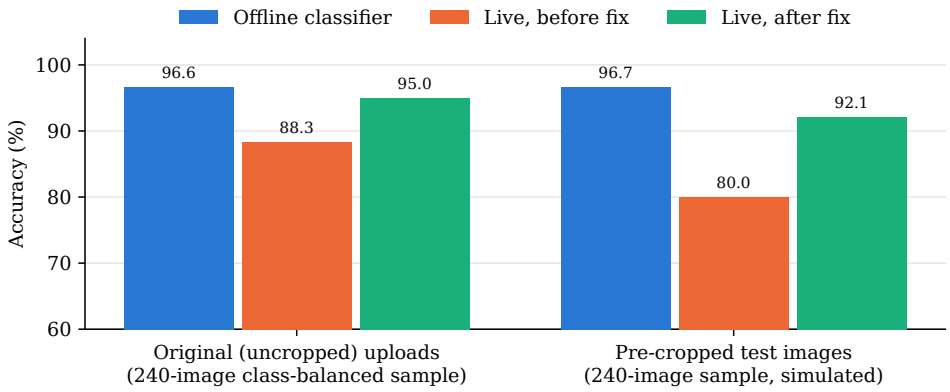


Figure 7.5: X-ray accuracy offline and through the live route, before and after the crop fix (the offline bar for original uploads is the full-test-split reference)

After the fix, the complete X-ray test split (1,922 images) was sent through the live API with every guard active. Figure 7.6 shows the result: 1,909 images were served and 13 were rejected

by the OOD detector (6 COVID-19 and 7 Tuberculosis). Counting the rejections as errors, live accuracy is 91.05% (91.67% over served images). The CT route shows no loss at all: the same 103 test images give 98.06% live, identical to offline, because the CT path has no crop step.

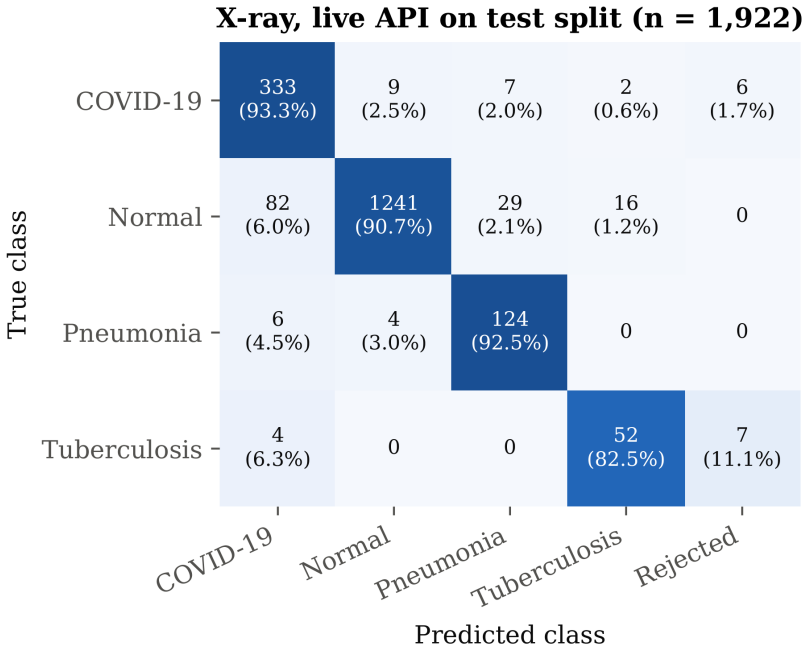


Figure 7.6: X-ray test split through the live API with all guards active (rejections shown as a column)

The remaining gap of about 5.5 points comes mainly from Normal images predicted as COVID-19 (82 of 1,368). We trace it to a class-dependent crop difference baked into the training data: COVID-19, Pneumonia and the COVID-source Normal images were cropped with the dataset's lung masks, whereas Tuberculosis and the TB-source Normal images, which have no masks, were cropped with a uniform border; at serving time every image gets the U-Net crop. Closing the gap requires rebuilding the training data with one crop convention and retraining. The finding also shows the limit of the start-up check: it replays the checkpoint on the same pre-processed validation data and so cannot see a defect in the serving path. Only a test through the real route found it.

7.4.1 Segmentation-Crop Trade-off

An earlier study, run on the previous ImageNet-backbone X-ray model with a 60-image stratified sample, compared the U-Net crop with a uniform border crop. The U-Net crop raised overall accuracy from 88.3% to 95.0%, but lowered Tuberculosis accuracy from 14/15 to 13/15. The TB source has no masks, so TB training images were built only with the border crop, and the tighter segmentation crop shifts live TB inputs slightly away from that. We accept this disclosed trade-off for the net gain of 6.7 points. The same class-dependent framing effect is still present with the current model, although it is much smaller than the geometric defect corrected by the crop fix.

7.5 Internal vs. External Accuracy

The final tests checked the classifiers beyond their validation splits: new images from sources outside the training databases were sent through the deployed routes with every guard active. Table 7.4 and Figure 7.7 compare the internal (validation) accuracy of each classifier with its accuracy on these external images.

Table 7.4: Internal (validation) vs. external detection accuracy

Modality	Internal (validation images)	External (new images)
Chest X-ray	96.5%	93.9% (31 of 33)
Lung CT	99.0%	91.9%

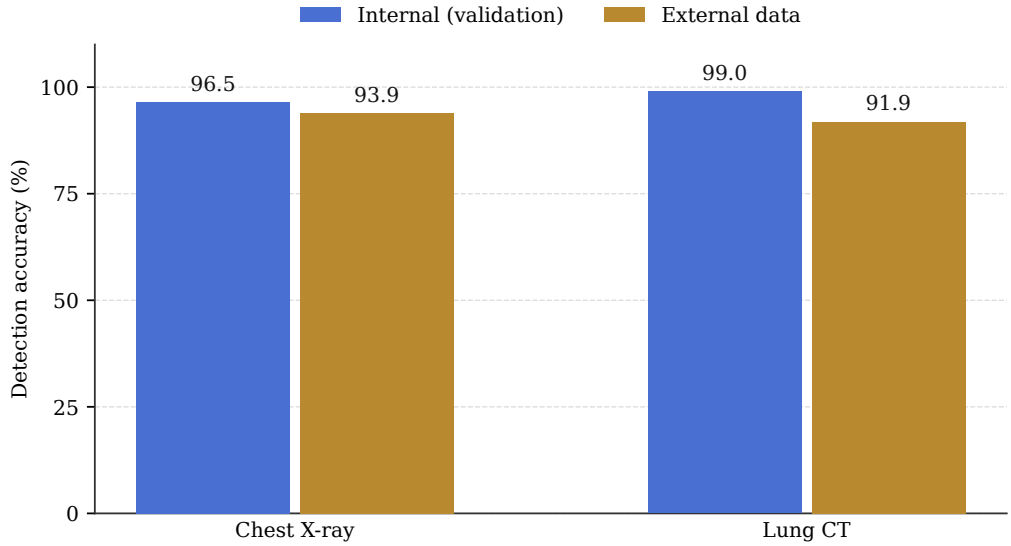


Figure 7.7: Detection accuracy on internal validation images and on new external images

X-ray. Of the external radiographs, 33 were accepted by the guards and 31 of them were classified correctly, an accuracy of 93.9% (internal validation: 96.5%).

CT. On external CT slices the classifier reaches 91.9% (internal validation: 99.0%). Both classifiers therefore maintain high detection accuracy on images from outside their training databases.

7.6 Snapshots of the Application

Figures 7.8 to 7.13 show the current version of the deployed application; further screens are in the appendix.

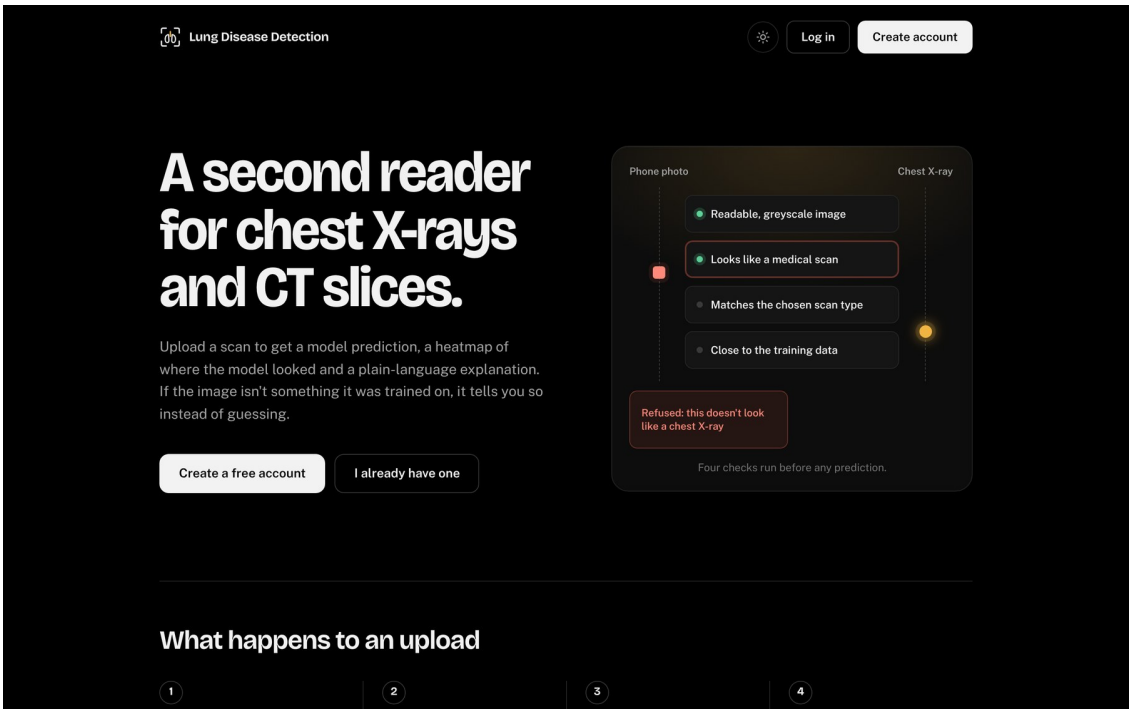


Figure 7.8: Landing page showing the four checks every upload passes before it is classified

The landing page describes the tool as a second reader for chest X-rays and CT slices, shows the four input checks (readable greyscale image, looks like a medical scan, matches the chosen scan type, close to the training data), and explains what happens to an upload.

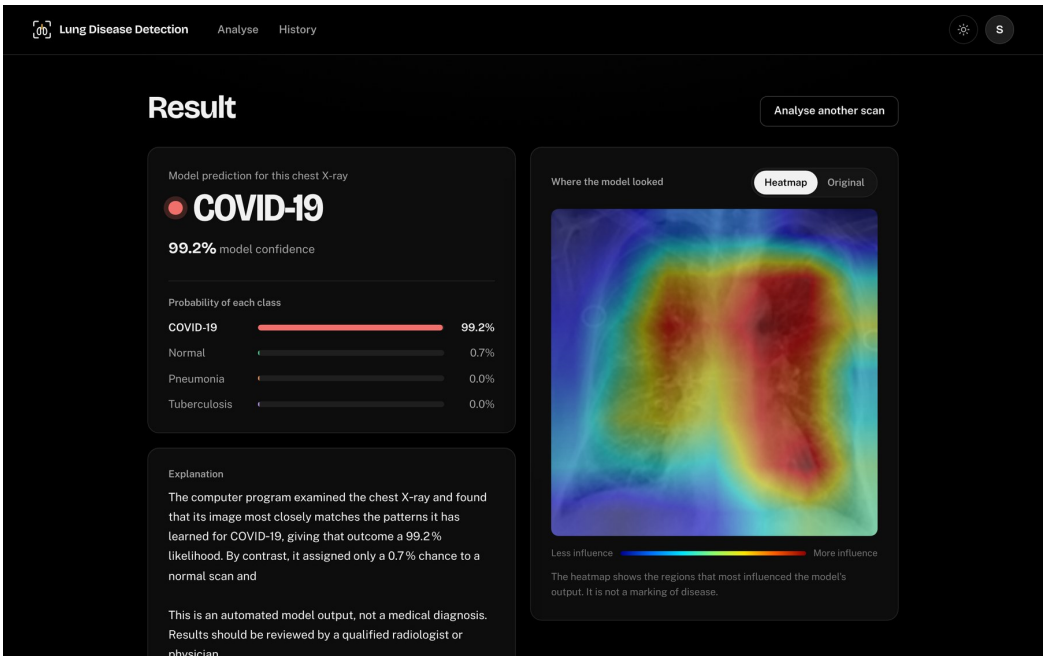


Figure 7.9: X-ray result: predicted class, probability bars, Grad-CAM heatmap and plain-language explanation

Figure 7.9 shows a COVID-19 prediction at 99.2% confidence. The probability bars show the full distribution rather than just the top class, the heatmap marks the regions that most influenced the

prediction, and the explanation is written in plain language with the non-diagnostic disclaimer. The Heatmap/Original toggle switches the image without another request.

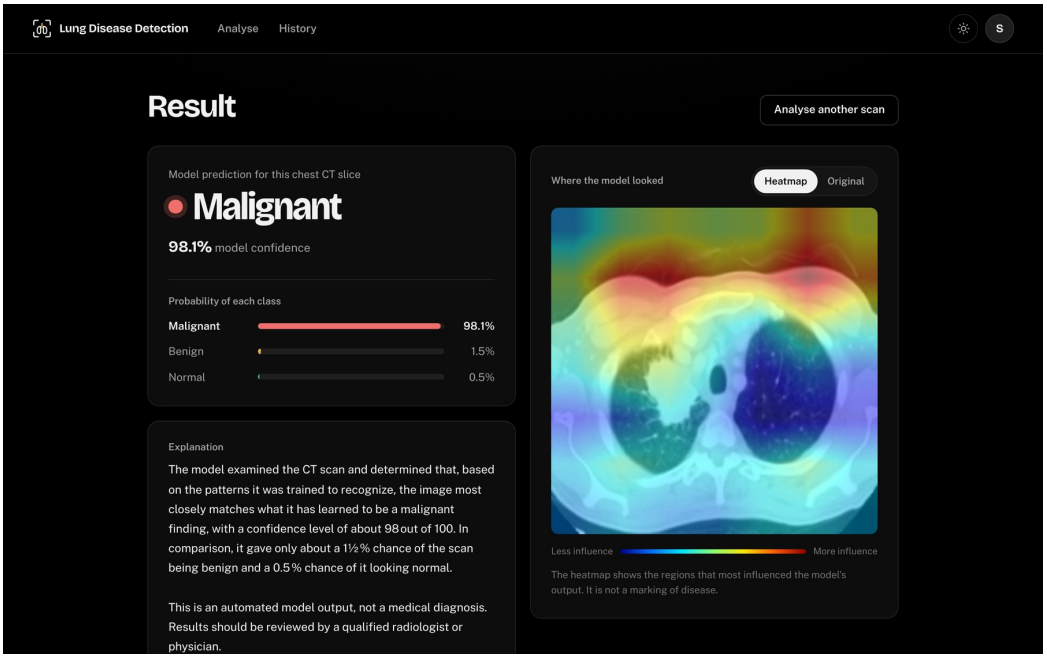


Figure 7.10: CT result for a Malignant prediction, in the same layout as the X-ray result

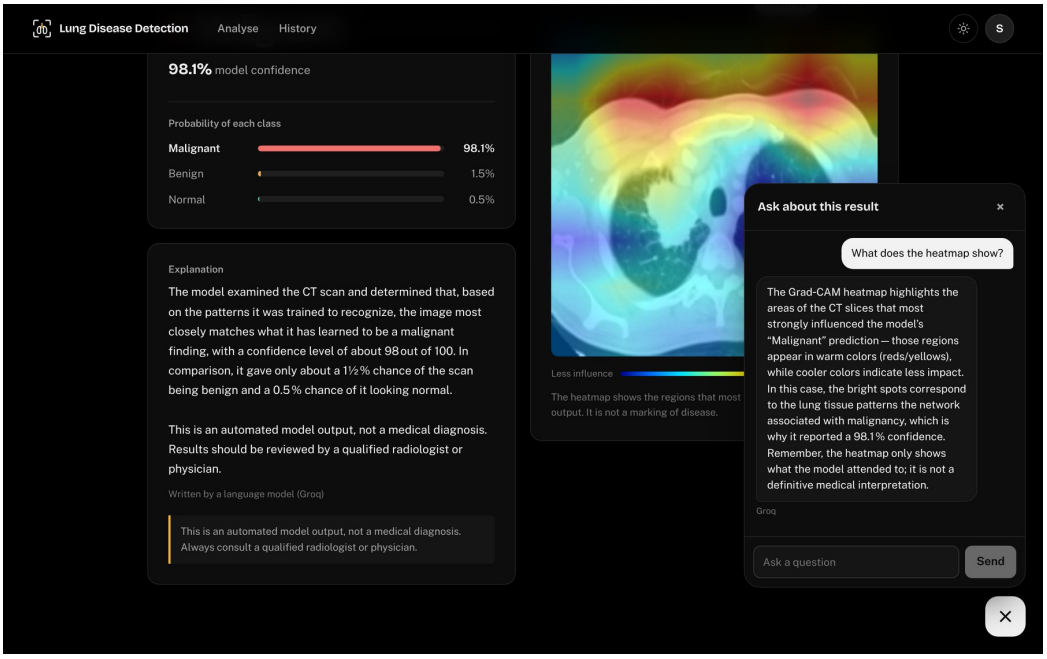


Figure 7.11: Chat assistant answering "What does the heatmap show?"

The chat assistant (Figure 7.11) explains what Grad-CAM shows and that the heatmap indicates where the model "looked", not a medical interpretation.

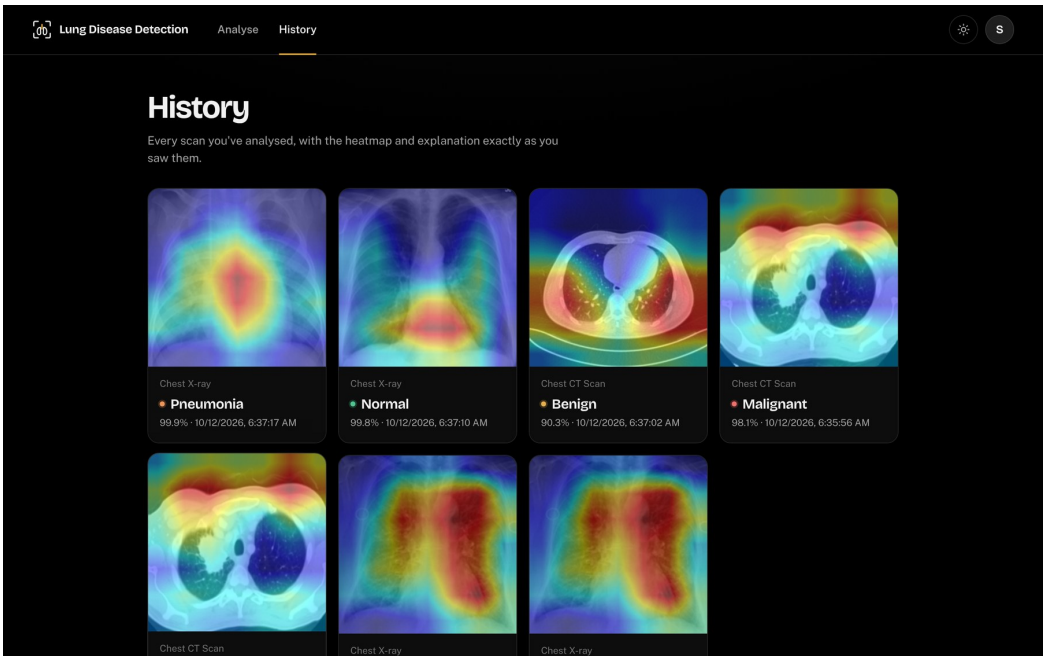


Figure 7.12: History page with every saved scan

Each history card (Figure 7.12) reproduces the modality, class, confidence, heatmap and time saved at prediction time; opening a card shows the full explanation.

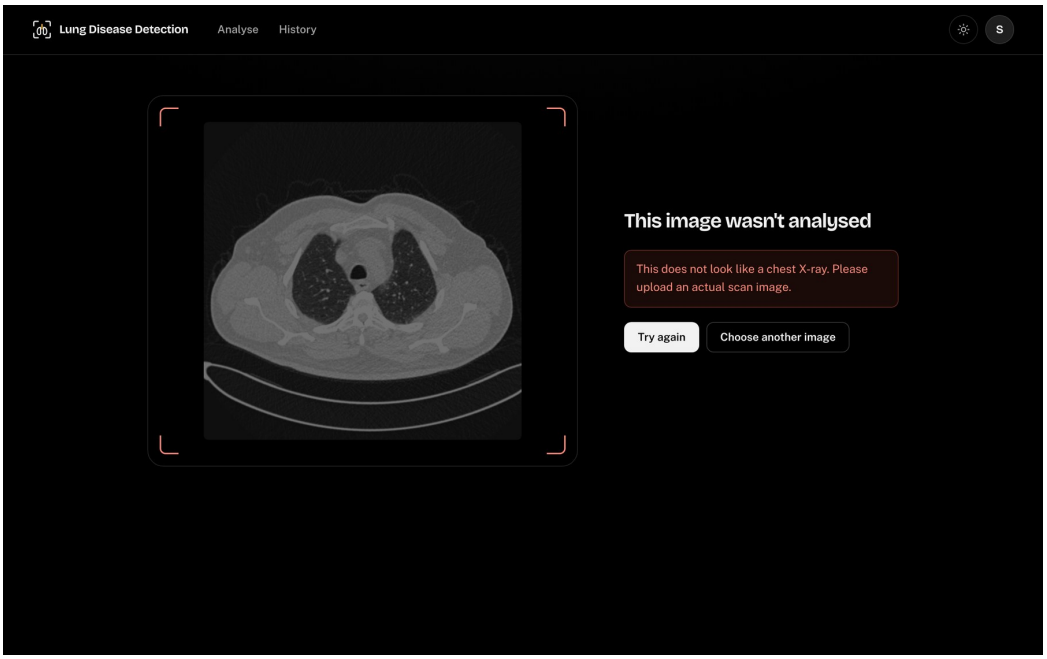


Figure 7.13: A guard firing live: a CT slice uploaded on the X-ray tab is refused with a reason

Figure 7.13 shows the guard layer at work: a CT slice uploaded on the X-ray tab is refused before any disease model runs, the user is told that the image does not look like a chest X-ray, and *Try again* and *Choose another image* buttons are offered.

7.7 Discussion

What the results show. The system meets its functional objectives. It classifies both modalities with high accuracy (internal validation 96.5% X-ray and 99.0% CT; held-out test 96.57% and 98.06%; external images 93.9% and 91.9%), explains every prediction, rejects non-scans, wrong-modality uploads and unfamiliar images, and delivers all of this through a secure multi-user web application. The live-serving audit shows that the deployed X-ray route delivers 91.05% rather than the offline 96.57%; this gap has been measured, explained and partly closed, rather than hidden behind the offline figure.

What the results do not show. They do not show that the models are ready for clinical use. The external sets are small, and the remaining limitations trace back to the public training data: single-source classes, a paediatric-only Pneumonia class and a single-institution CT dataset. They are limitations of the data rather than of the software.

Limitations.

- The CT dataset is small (1,036 images; 103 in test) and from one institution; single-image differences move accuracy by about one point.
- Every Pneumonia training image is paediatric, and COVID-19 and Tuberculosis each come from a single source database, so the X-ray classes are confounded with their sources.
- The external sets are small and do not cover every class equally, so the external accuracy figures have wide confidence intervals.
- The gates were trained on cropped training images but receive uncropped uploads; the effect of this on their rejection rates has not been measured.
- The OOD detector does not catch genuine chest images of diseases outside the label set (for example Lung Opacity).
- The segmentation U-Net was trained outside the repository, and no training script for it exists in the project.
- The hosted deployment runs with the start-up reproduction check switched off, because the validation data are not shipped with it.
- The prediction and chat APIs accept anonymous calls (bounded by rate limits), sessions cannot be revoked before expiry, rate limiting is in-memory and per process, and SQLite allows one writer, so a multi-server deployment would need changes.

CHAPTER 8

CONCLUSIONS AND FUTURE SCOPE

8.1 Conclusion

This project designed, built and evaluated the AI-Powered Lung Disease Detection System, a web application that classifies chest X-rays into COVID-19, Normal, Pneumonia or Tuberculosis and chest CT slices into Benign, Malignant or Normal. Both classifiers are DenseNet-121 networks trained by transfer learning: the X-ray model from chest-radiograph weights and the CT model from ImageNet weights, each fine-tuned with a class-weighted loss to handle severe class imbalance. On their internal validation data they reach 96.5% and 99.0% accuracy (96.57% and 98.06% on the held-out test splits), and on new external images 93.9% and 91.9%.

Around the classifiers, we implemented the features that make a model usable and safe to demonstrate. A guard layer of input checks, a per-modality sanity gate, a modality gate and a feature-space out-of-distribution detector rejects photographs, documents, screenshots, wrong-modality scans and unfamiliar images with a plain-English reason instead of forcing them into a disease class. A U-Net crops X-rays to the lung fields so that the classifier sees what it was trained on. Every prediction comes with its full probability vector, a Grad-CAM heatmap, a low-confidence warning where appropriate, a non-diagnostic plain-language report and a scoped chat assistant. The application provides registration with email verification, Google sign-in, password reset, scan history and rate limiting, and it is deployed publicly over HTTPS in Docker containers on an AWS EC2 instance. In strict mode it refuses to start unless all six model checkpoints reproduce their claimed validation metrics, and an automated suite of 81 tests covers the back end.

The project's most transferable lesson is about evidence. Auditing the data turned an implausible 100% CT accuracy into an honest 98.06% and removed a burned-in artifact from the tuberculosis images. Testing the deployed route rather than the offline model found a preprocessing mismatch that had cut live X-ray accuracy by up to 19 points, and fixing it brought the live figure to 91%. On new external images sent through the deployed routes, the X-ray route reached 93.9% (31 of 33 accepted images) against 96.5% internally, and the CT route 91.9% against 99.0%. We conclude that measuring a medical AI system through the path a user actually takes, and on data it has never seen, is as important as training it.

8.2 Future Scope

The work can be extended in the following directions:

- **Better X-ray data.** Rebuild the Pneumonia class with adult radiographs (for example the RSNA pneumonia images behind the excluded Lung Opacity class), draw every class from several sources, and rebuild the training data with a single crop convention to close the remaining live-serving gap.
- **Better CT data.** Train on multi-institution CT data with patient identifiers, such as LIDC-IDRI, so that splits can be made by patient and the model cannot rely on one hospital's acquisition style.
- **Larger external validation.** Extend the external evaluation to hundreds of independently sourced, provenance-checked images per class for both modalities, with every class represented.
- **Stronger guards and calibration.** Train the gates on uncropped inputs, add an explicit "other abnormality" class or open-set method for in-scope but untrained findings such as lung opacity, and calibrate the confidence scores (for example with temperature scaling) so that a reported 90% means 90%.
- **Clinical evaluation.** Compare the system's output and heatmaps with radiologists' readings in a supervised study, and gather feedback on the usefulness of the reports.
- **Volumetric and DICOM support.** Accept DICOM files and full CT series.
- **Production hardening.** Require authentication on all prediction routes, add revocable sessions, move rate limiting to a shared store and the database to a server such as PostgreSQL, run the reproduction check in the hosted deployment as a background task, run the test suite in continuous integration, and measure throughput under load.

REFERENCES

[1] Pranav Rajpurkar, Jeremy Irvin, Kaylie Zhu, Brandon Yang, Hershel Mehta, Tony Duan, Daisy Ding, Aarti Bagul, Curtis Langlotz, Katie Shpanskaya, Matthew P. Lungren, and Andrew Y. Ng. CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning. *arXiv preprint arXiv:1711.05225*, 2017.

[2] Jeremy Irvin, Pranav Rajpurkar, Michael Ko, Yifan Yu, Silviana Ciurea-Ilcus, Chris Chute, Henrik Marklund, Behzad Haghgoo, Robyn Ball, Katie Shpanskaya, et al. CheXpert: A large chest radiograph dataset with uncertainty labels and expert comparison. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 590–597, 2019.

[3] Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4700–4708, 2017.

[4] Joseph Paul Cohen, Joseph D. Viviano, Paul Bertin, Paul Morrison, Parsa Torabian, Matteo Guarrera, Matthew P. Lungren, Akshay Chaudhari, Rupert Brooks, Mohammad Hashir, and Hadrien Bertrand. TorchXRayVision: A library of chest X-ray datasets and models. In *Proceedings of the International Conference on Medical Imaging with Deep Learning (MIDL)*, volume 172 of *Proceedings of Machine Learning Research*, pages 231–249, 2022.

[5] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255, 2009.

[6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[7] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[8] World Health Organization. Global tuberculosis report 2025. Technical report, World Health Organization, Geneva, 2025.

[9] N. Ramnath, P. Ganesan, P. Penumadu, D. Arenberg, and A. Bryant. Lung cancer screening in India: Preparing for the future using smart tools & biomarkers. *Indian Journal of Medical Research*, 160(6):561–569, 2025.

[10] R. Arora. The training and practice of radiology in India: Current trends. *Quantitative Imaging in Medicine and Surgery*, 4(6):449–450, 2014.

[11] United Nations. Transforming our world: The 2030 agenda for sustainable development. United Nations General Assembly, A/RES/70/1, 2015.

[12] Linda Wang, Zhong Qiu Lin, and Alexander Wong. COVID-Net: A tailored deep convolutional neural network design for detection of COVID-19 cases from chest X-ray images. *Scientific Reports*, 10(1):19549, 2020.

[13] Muhammad E. H. Chowdhury, Tawsifur Rahman, Amith Khandakar, Rashid Mazhar, Muhammad Abdul Kadir, Zaid Bin Mahbub, Khandakar Reajul Islam, Muhammad Salman Khan, Atif Iqbal, Nasser Al Emadi, Mamun Bin Ibne Reaz, and Mohammad Tariqul Islam. Can AI help in screening viral and COVID-19 pneumonia? *IEEE Access*, 8:132665–132676, 2020.

[14] Tawsifur Rahman, Amith Khandakar, Muhammad Abdul Kadir, Khandaker Rejaul Islam, Khandakar Fahad Islam, Rashid Mazhar, Tahir Hamid, Mohammad Tariqul Islam, Saad Kashem, Zaid Bin Mahbub, Mohamed Arselene Ayari, and Muhammad E. H. Chowdhury. Reliable tuberculosis detection using chest X-ray with deep learning, segmentation and visualization. *IEEE Access*, 8:191586–191601, 2020.

[15] Alex Mirugwe, Lillian Tamale, and Juwa Nyirenda. Improving tuberculosis detection in chest X-ray images through transfer learning and deep learning: Comparative study of convolutional neural network architectures. *JMIRx Med*, 6:e66029, 2025.

[16] Patrice Monkam, Shouliang Qi, He Ma, Weiming Gao, Yudong Yao, and Wei Qian. Detection and classification of pulmonary nodules using convolutional neural networks: A survey. *IEEE Access*, 7:78075–78091, 2019.

[17] Zahia Benamara, Soraya Zehani, Athmane Zitouni, and Tawfiq Beghriche. An innovative multi-level fine-tuning deep learning approach for enhanced lung cancer classification. *Network Modeling Analysis in Health Informatics and Bioinformatics*, 14:90, 2025.

[18] Hamdalla Alyasriy and Muayed Al-Huseiny. The IQ-OTH/NCCD lung cancer dataset. Mendeley Data, v1, 2020. `https://www.kaggle.com/datasets/hamdallak/the-iqothnccd-lung-cancer-dataset`.

[19] Dan Hendrycks and Kevin Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1610.02136.

[20] Ting Liu, Mengmeng Song, Shanshan Tu, and Lei Xu. A survey on automatic generation of medical imaging reports based on deep learning. *BioMedical Engineering OnLine*, 22(1):134, 2023.

[21] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, 9th edition, 2019.

[22] M. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). RFC 7519, Internet Engineering Task Force, 2015.

[23] Muhammad E. H. Chowdhury, Tawsifur Rahman, Amith Khandakar, et al. COVID-19 radiography database. Kaggle, 2021. `https://www.kaggle.com/datasets/tawsifurrahman/covid19-radiography-database`.

[24] Tawsifur Rahman, Amith Khandakar, et al. Tuberculosis (TB) chest X-ray database. Kaggle, 2021. `https://www.kaggle.com/datasets/tawsifurrahman/tuberculosis-tb-chest-xray-dataset`.

[25] Daniel S. Kermany, Michael Goldbaum, Wenjia Cai, et al. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5):1122–1131, 2018.

[26] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[27] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[28] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[29] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.

[30] Sebastián Ramírez. FastAPI framework documentation. `https://fastapi.tiangolo.com`. Accessed: 2026.

[31] Meta Open Source. React documentation. `https://react.dev`. Accessed: 2026.

APPENDIX A

SUPPLEMENTARY MATERIAL

A.1 Screenshots

The following screens were captured from the current version of the running application; further screens appear in Section 7.6.

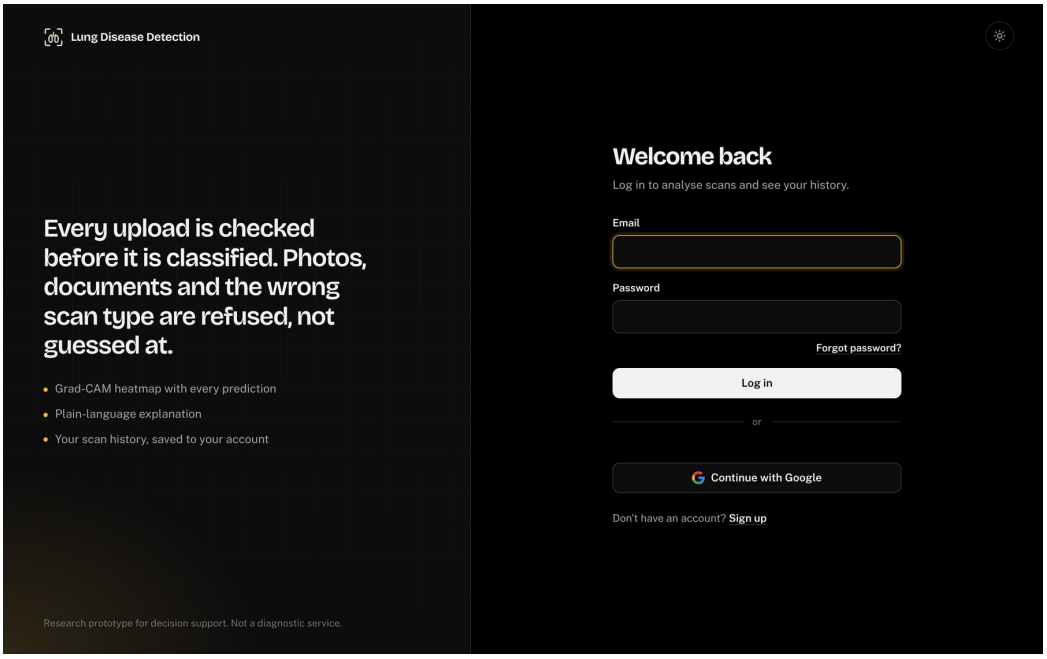


Figure A.1: Login screen, with email and password or “Continue with Google”

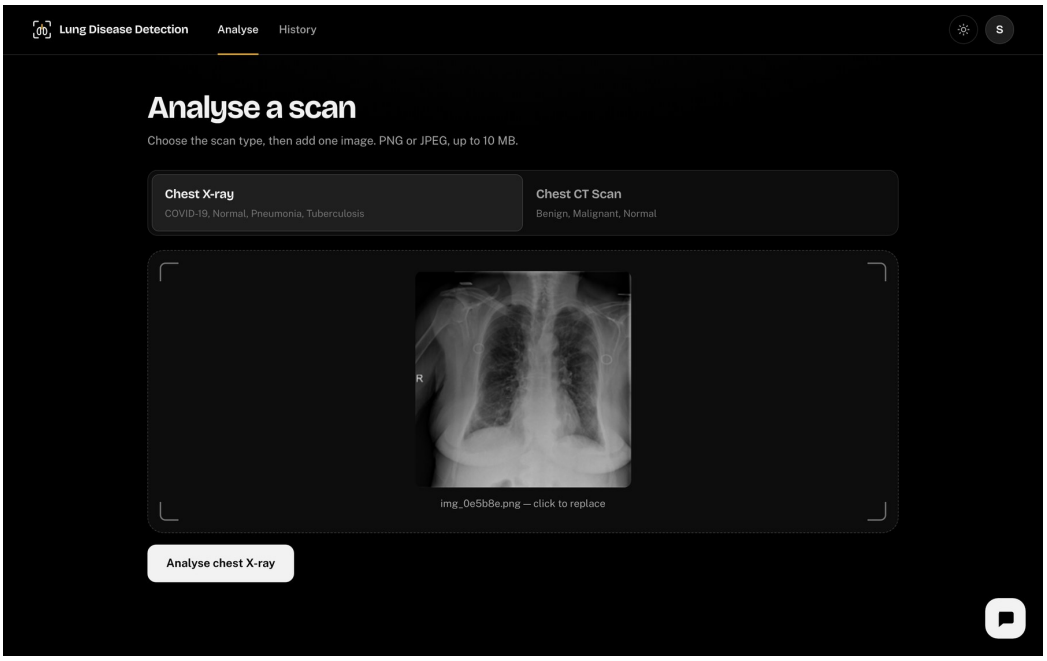


Figure A.2: Analyse screen: choosing the scan type and uploading an image

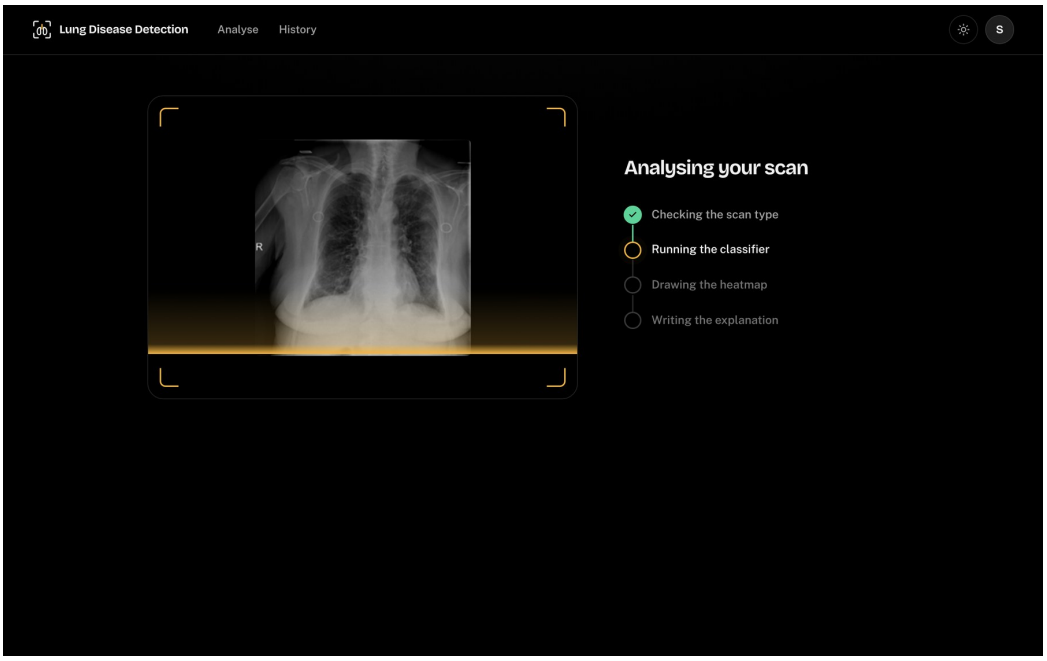


Figure A.3: Analysing screen with the stage checklist

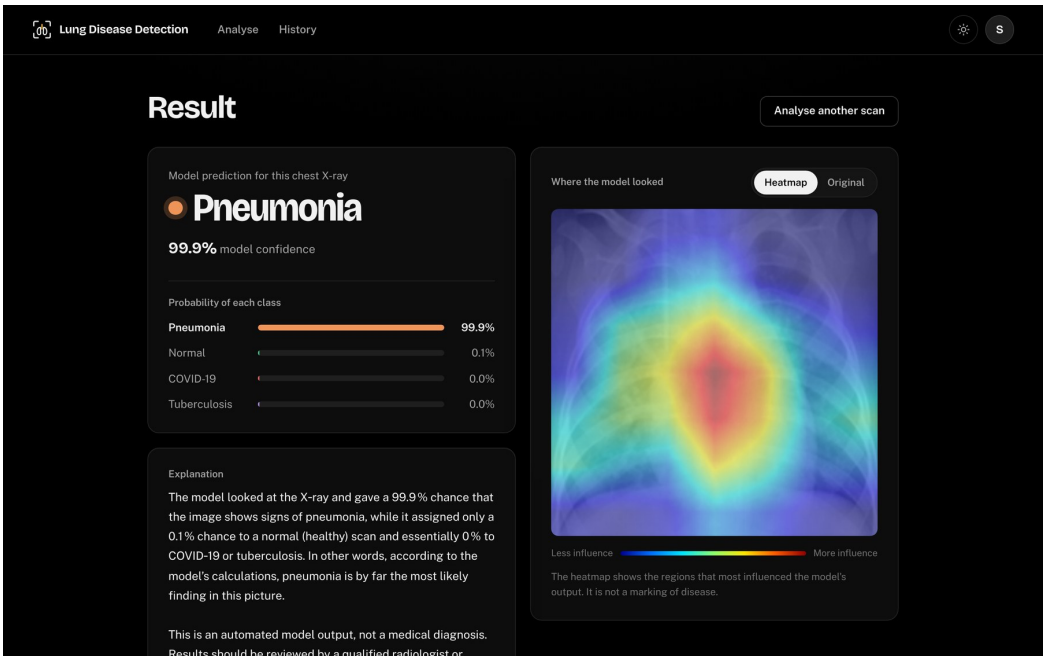


Figure A.4: X-ray result with Grad-CAM heatmap (Pneumonia prediction)

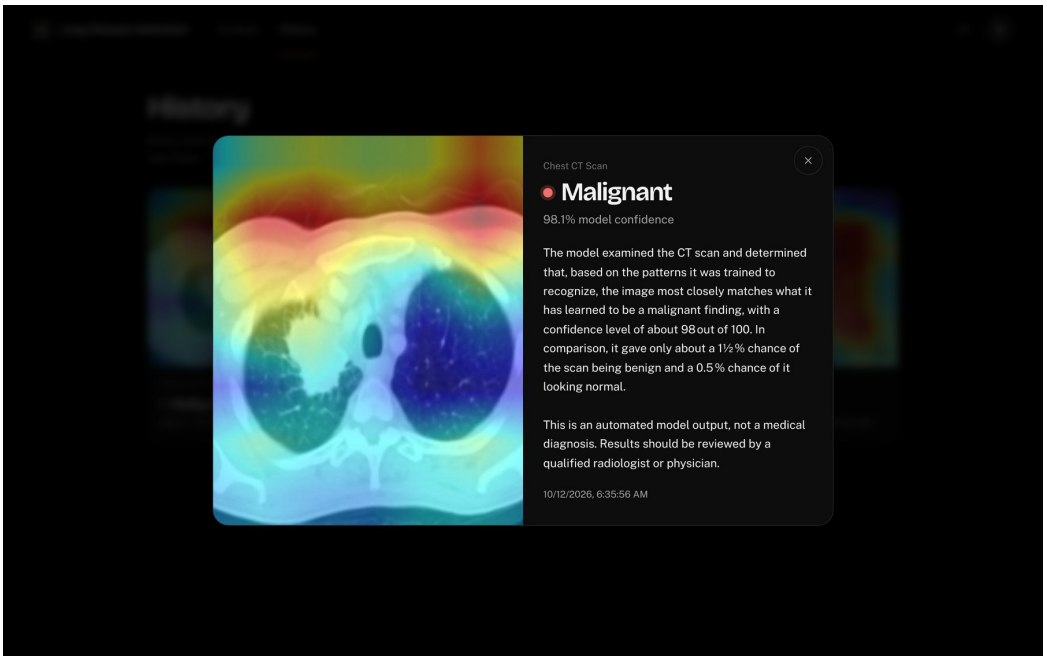


Figure A.5: An opened history entry showing the saved heatmap and explanation

A.2 Gantt Chart

Figure A.6 shows the project timeline. The dashed lines mark Review 1 (7 September), Review 2 (15 October) and Review 3 (14 November).

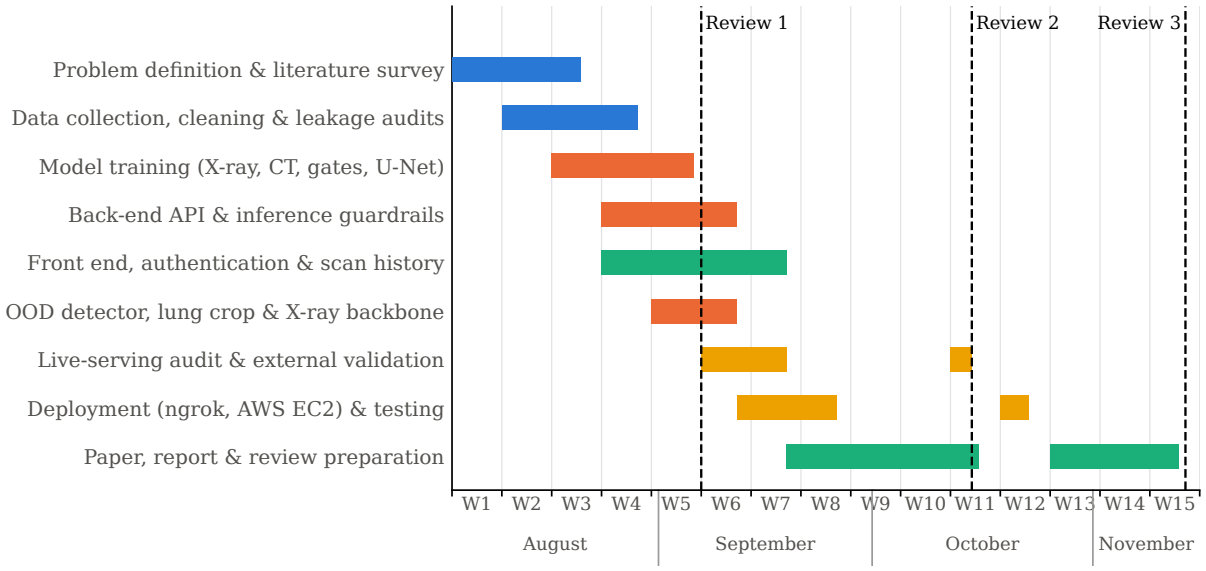


Figure A.6: Gantt chart of the project

A.3 Glossary

Table A.1: Glossary of terms

Term	Meaning
AUC	Area under the ROC curve; 0.5 is chance and 1.0 is perfect separation.
Bcrypt	A deliberately slow password-hashing function with a built-in salt.
CNN	Convolutional neural network, a deep network suited to images.
CT	Computed tomography; a set of cross-sectional X-ray images of the body. This project uses single 2D slices.
DenseNet-121	A 121-layer CNN in which each layer receives the feature maps of all earlier layers in its block.
Dice / IoU	Overlap measures between a predicted and a true segmentation mask.
External validation	Testing a model on data from sources not used in training.
F1-score	Harmonic mean of precision and recall.
Grad-CAM	Gradient-weighted Class Activation Mapping; a heatmap of the image regions that most influenced a prediction.
JWT	JSON Web Token; a signed, self-contained session token.
Leakage	Near-identical data present in both training and test sets, which inflates test accuracy.
LLM	Large language model, used here to write reports and chat replies.

Term	Meaning
Modality	The type of imaging test, here chest X-ray or chest CT.
OAuth 2.0	A standard that lets users sign in with an existing account such as Google.
OOD	Out-of-distribution; an input unlike the data a model was trained on.
Precision / Recall	Fraction of predictions of a class that are correct / fraction of true cases of a class that are found.
Sanity gate	A classifier that decides whether an upload is a medical scan at all.
Shortcut learning	A model relying on incidental cues (markers, scanner style, source) rather than the intended signal.
SSIM	Structural Similarity Index; a measure of how similar two images look.
Transfer learning	Starting from a model pre-trained on a large dataset and fine-tuning it on a smaller one.
U-Net	An encoder-decoder CNN with skip connections, widely used for medical image segmentation.

A.4 Description of Tools and Technologies Used

Table A.2: Tools and technologies used

Tool / technology	Use in the project
Python 3.12	Language for the models, data scripts and back end.
PyTorch, torchvision	Deep-learning framework [29] used to define, train and run the models; torchvision supplies DenseNet-121, ResNet-18 and image transforms.
TorchXRayVision	Library of chest X-ray models and datasets [4]; provides the pre-trained DenseNet-121 used for the X-ray classifier and its preprocessing.
OpenCV, Pillow, scikit-image, ImageHash	Image loading, cropping, inpainting, colour maps, perceptual hashing and SSIM for the data audits and Grad-CAM overlays.
NumPy, SciPy, scikit-learn, matplotlib	Numerical work, metrics, the logistic-regression source probe and plots.
FastAPI, Uvicorn, Pydantic	Web framework [30], ASGI server and request/response validation for the REST API, with automatic OpenAPI documentation.
slowapi	Per-client rate limiting.
bcrypt, PyJWT	Password hashing and session tokens.
SQLite	Embedded relational database for users, tokens and scans.

Tool / technology	Use in the project
Groq API	Low-latency hosting of the `openai/gpt-oss-120b` LLM for reports and chat.
Gmail SMTP	Delivery of verification and password-reset emails.
Google OAuth 2.0	"Continue with Google" sign-in.
React 19, React Router, Vite	Front-end library [31], client-side routing and build tool / development server.
pytest, httpx, Playwright	Automated back-end tests and browser automation for screenshots.
PlantUML	UML modelling tool used for the use case, class, sequence, state and design-pattern diagrams; the diagram sources are kept in `docs/uml/` next to the code.
Docker, Docker Compose	Packaging of the application into an image and running the hosted services.
Caddy	Reverse proxy that terminates HTTPS with automatically renewed Let's Encrypt certificates.
AWS EC2	Cloud server that hosts the public deployment.
ngrok	HTTPS tunnel for local public demonstrations.
Google Colab	Cloud GPU for full training runs.
Git, VS Code	Version control and editor.
LaTeX (Overleaf)	Preparation of this report.

A.5 Blueprint

The project blueprint is enclosed at the end of this report.

A.6 Plagiarism Report

The plagiarism report is enclosed at the end of this report.